



especialização

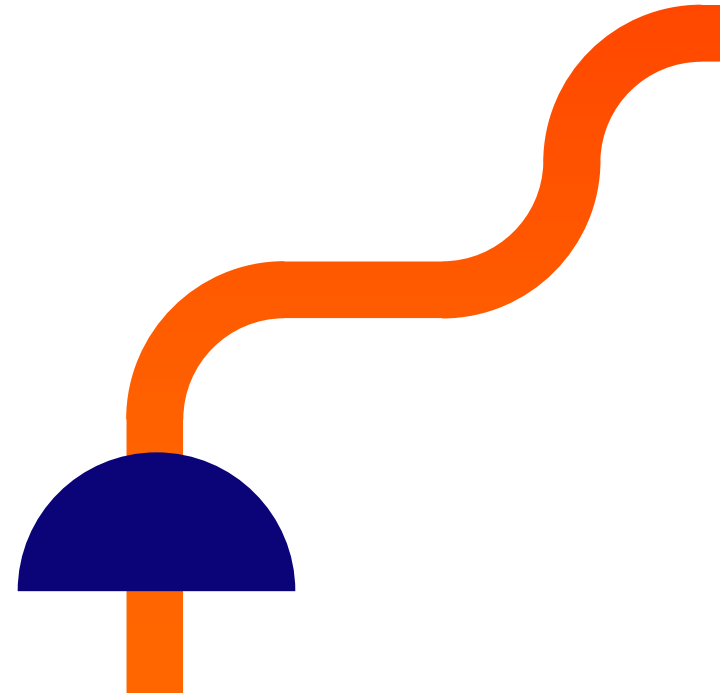
Aplicações de IA e ML em RIC

Julio Tesolin

Open RAN - Redes Abertas e Tecnologias Emergentes

AGENDA DA AULA

- 1) ML – Redes Neurais: RNN, LSTM e Autoencoders
- 2) ML – Aprendizado por Reforço





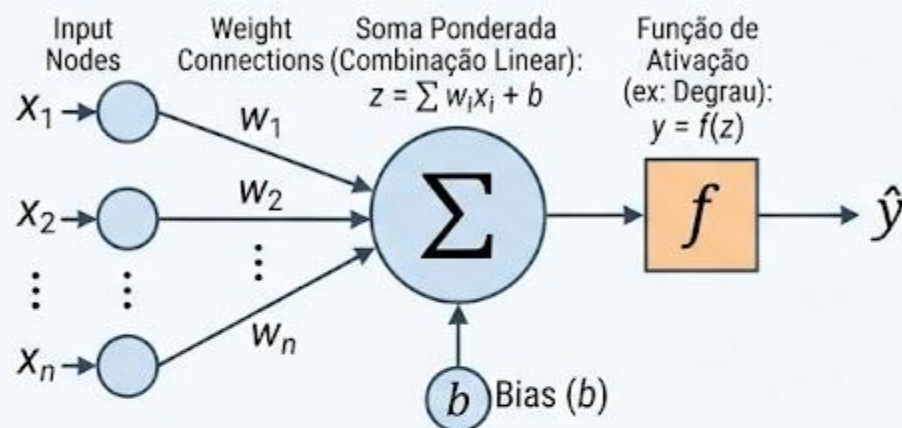
especialização

Aplicações IA/ML RIC: Breve Revisão Aula Anterior

Julio Tesolin

Open RAN Redes Abertas e Tecnologias Emergentes

O Perceptron: Diagrama e Conceitos Fundamentais



Elementos do Perceptron

-  **Entradas (x_i):** Dados de entrada (ex: características da amostra).
-  **Pesos (w_i):** Importância atribuída a cada entrada.
-  **Soma Ponderada:** Combinação linear das entradas e pesos (z).
-  **Função de Ativação:** Aplica uma não-linearidade para produzir a saída.
-  **Saída ($\hat{y}$):** Resultado final da classificação (0 ou 1).

1. Processo de Aprendizado (Regra Perceptron)

-  **Inicialização:** Pesos e viés são inicializados aleatoriamente ou com zeros.
-  **Cálculo da Saída:** Para cada exemplo de treinamento, o Perceptron calcula a saída prevista ($\hat{y}$).
-  **Atualização de Pesos:** Os pesos são ajustados pelo erro (Target - $\hat{y}$) usando a Taxa de Aprendizado (η) ($\mathbf{w} \leftarrow \mathbf{w} + \eta(\mathbf{T} - \hat{\mathbf{y}})\mathbf{x}_i$) para reduzir o erro.

2. Vantagens sobre ML Tradicional

-  **Aprendizagem Online:** Capaz de aprender incrementalmente com novos dados (instância por instância).
- **Simplicidade Computacional:** Estrutura leve e fácil de implementar para problemas linearmente separáveis.
-  **Base para Redes Profundas:** É o bloco fundamental de construção para Redes Neurais Multilayer Perceptron (MLP) e Deep Learning.

3. Limitações

- **Fronteiras Lineares Simples:** Incapaz de classificar dados não linearmente separáveis.
- **O Problema XOR:** Fundamentalmente não consegue resolver a função lógica XOR (Ou-Exclusivo).
- **Incapacidade Complexa:** Limitado a separar dados com uma única linha reta, falhando em padrões complexos.

BREVE REVISÃO

FORMULAÇÃO MATEMÁTICA

$$y = \sigma(z) = \sigma\left(\sum_{i=1}^n x_i w_i + b\right)$$

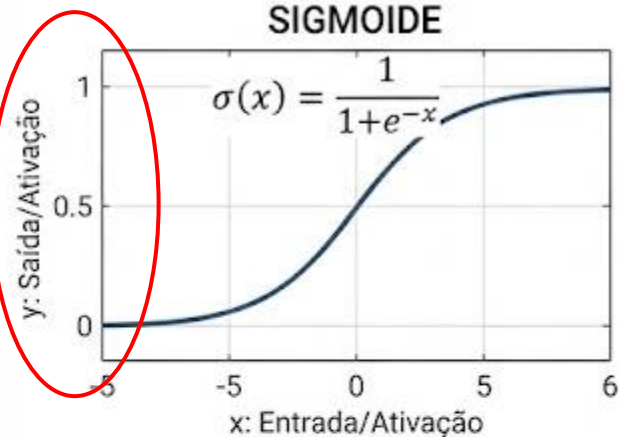
- ❑ x_i : entradas (dados recebidos)
- ❑ w_i : pesos sinápticos (importância de cada entrada)
- ❑ b : viés (bias) , valor independente somado ao final
- ❑ σ : função de ativação

$\sigma(z) = z \rightarrow$ regressão linear

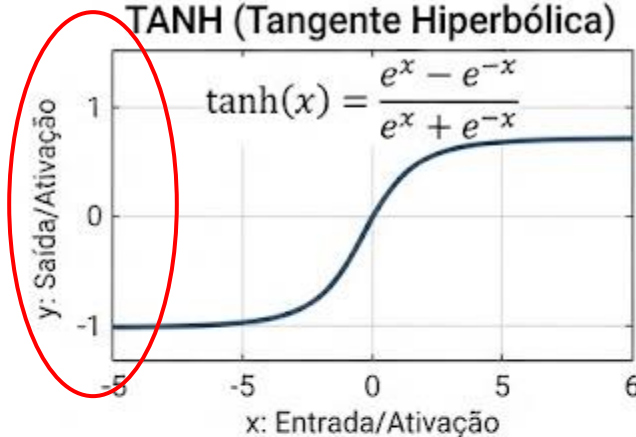
BREVE REVISÃO

FUNÇÕES DE ATIVAÇÃO PARA NEURÔNIOS ARTIFICIAIS

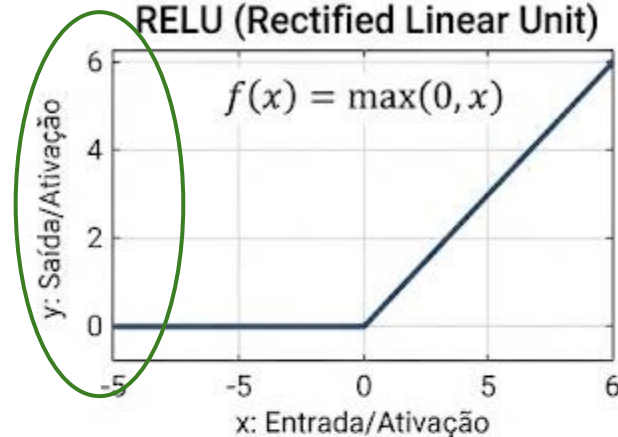
SIGMOIDE



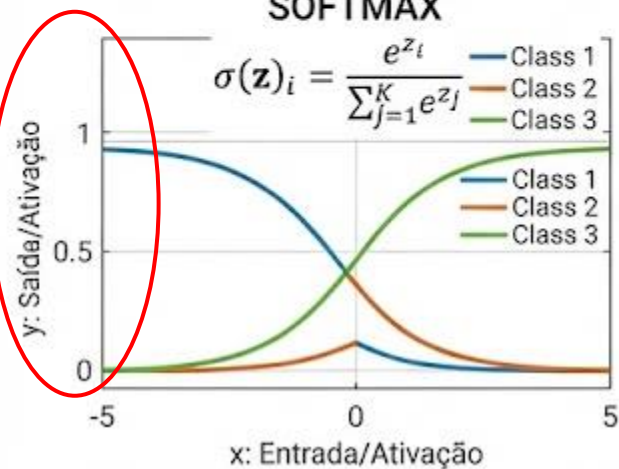
TANH (Tangente Hiperbólica)



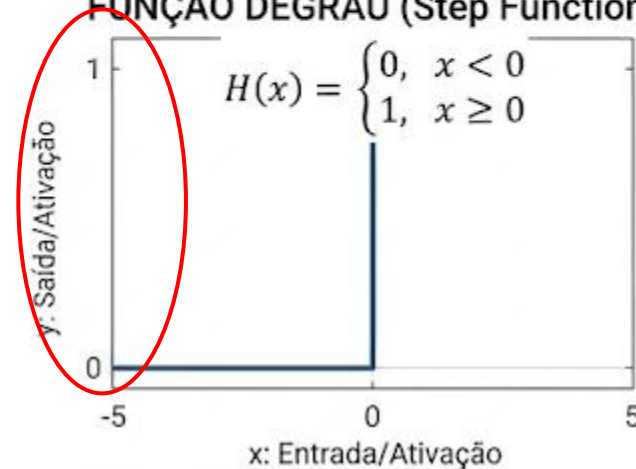
RELU (Rectified Linear Unit)



SOFTMAX



FUNÇÃO DEGRAU (Step Function)

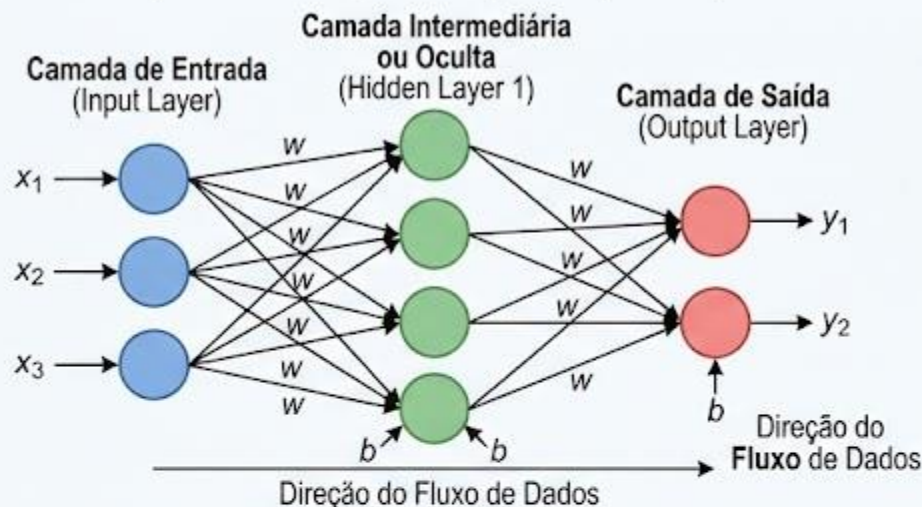


LINEAR POR PARTES



BREVE REVISÃO

O Multilayer Perceptron (MLP): Arquitetura e Conceitos



Elementos do MLP

-  **Camada de Entrada:** Recebe os dados brutos do problema. Número de neurônios = número de características.
-  **Camada Intermediária (Oculta):** Processa as informações da camada anterior, extraindo características e permitindo não-linearidades. Pode haver múltiplas camadas.
-  **Camada de Saída:** Produz o resultado final da rede (ex: classificação, regressão). Número de neurônios = número de classes ou saídas.
-  **Conexões e Pesos (w):** Definem a força da conexão entre neurônios, ajustados durante o treinamento.
-  **Bias (b):** Permite deslocar a função de ativação, essencial para o aprendizado.

Conceitos Chave

-  **Forward Propagation (Propagação Direta):** Dados de entrada passam pelas camadas para calcular a saída prevista.
-  **Backpropagation (Retropropagação):** O erro da saída é calculado e propagado de volta para ajustar os pesos e minimizar o erro.
-  **Função de Perda (Loss Function):** Mede a diferença entre a saída prevista e o valor real (erro).
-  **Gradiente (Gradient):** Vetor que aponta para a direção de maior aumento da função de perda; usado para guiar o ajuste de pesos.
-  **Taxa de Aprendizado (Learning Rate):** Hiperparâmetro que controla o tamanho dos passos na atualização dos pesos.

Vantagens e Limitações do MLP (em relação ao Perceptron Simples)

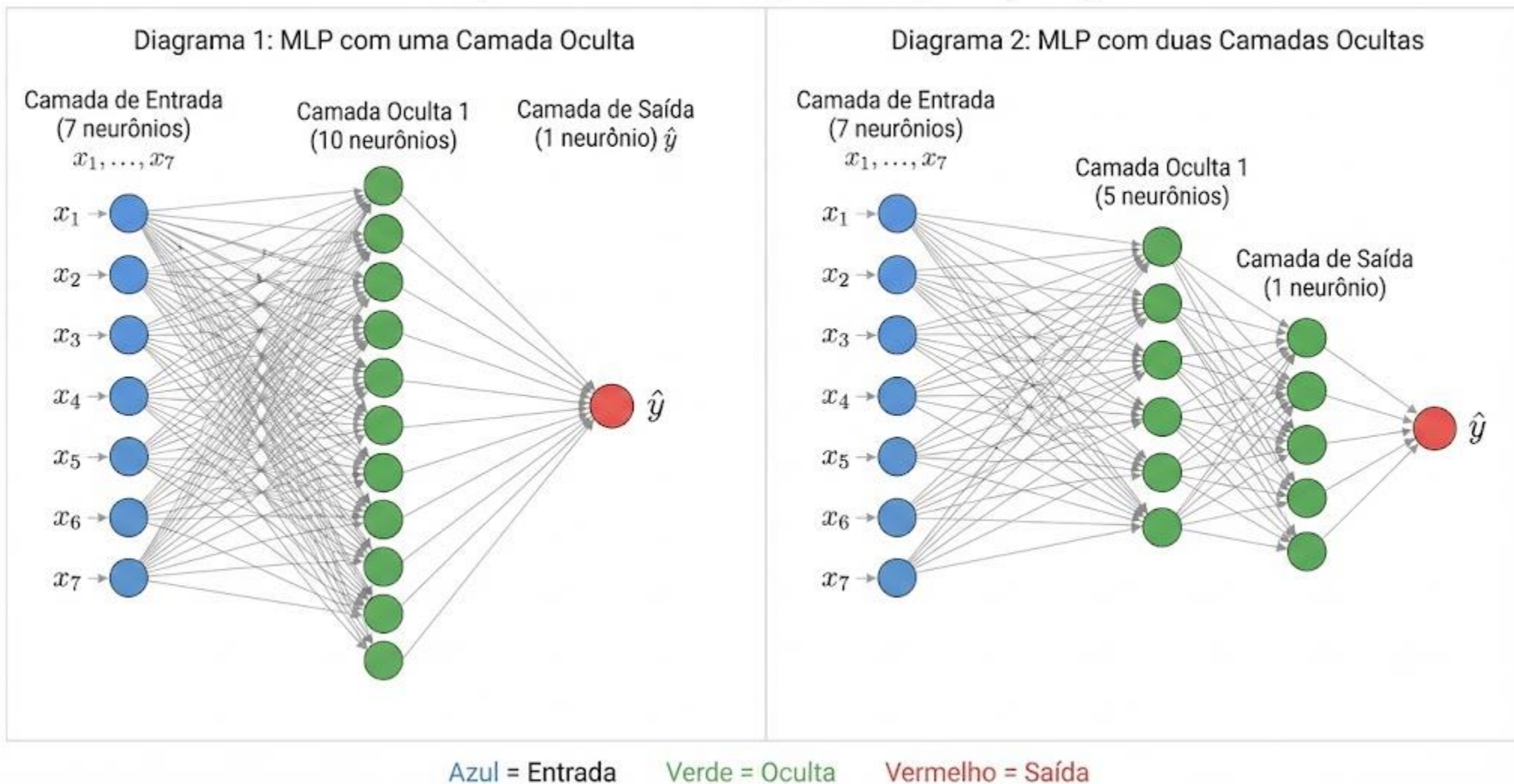
-  **Vantagens:** Resolve o problema do Perceptron (ex: fronteiras não-lineares, problema XOR) através de camadas ocultas e funções de ativação não-lineares.
-  **Vantagens:** Capaz de aprender representações complexas e modelos de alta dimensão.
-  **Limitações:** Pode exigir muitos dados e tempo de treinamento para convergência.
-  **Limitações:** Risco de overfitting, especialmente com redes muito profundas ou dados ruidosos.

NOTAS SOBRE MLPs

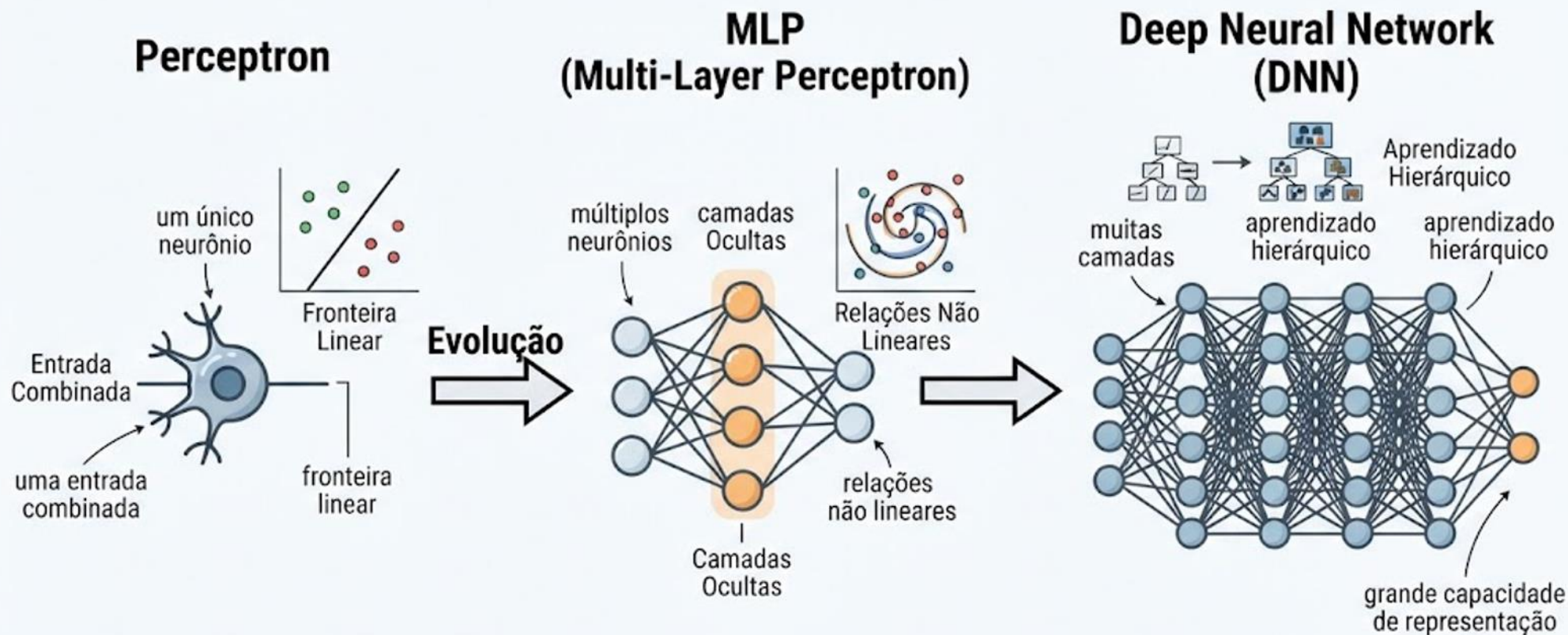
- ❑ Neurônios podem ter diferentes funções de ativação
- ❑ Padrão: neurônios de uma mesma camada tem a mesma função de ativação
- ❑ Qual função de ativação utilizar ?
 - Camadas ocultas: capturar padrões não lineares. **ReLU**, **Sigmoide** ou **Tangente Hiperbólica (tanh)** são essenciais para que a rede não se torne apenas uma simples operação linear
 - Camada de saída: escolha depende do tipo de resposta esperada (Regressão ou Classificação)

BREVE REVISÃO

Arquiteturas de Multilayer Perceptron (MLP)



BREVE REVISÃO



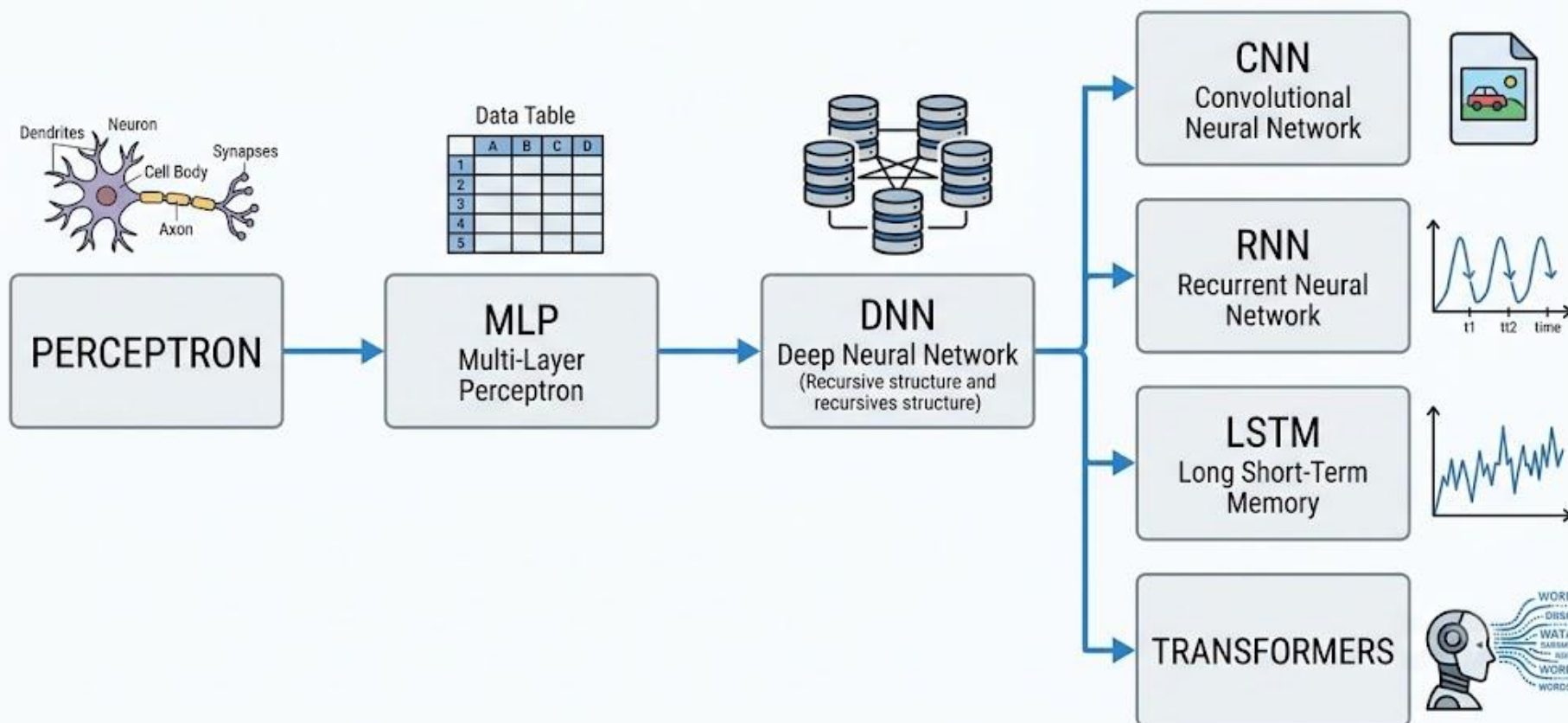
Maior complexidade → Maior capacidade de aprendizado

NOTAS SOBRE DNNs

- ❑ Toda MLP com muitas camadas pode ser considerada uma DNN, mas nem toda DNN é apenas uma MLP, pois as DNNs modernas utilizam "truques" arquitetônicos muito mais variados
- ❑ DNN é uma superclasse de outras arquitetura de rede neural (CNN, RNN, LSTM etc.)

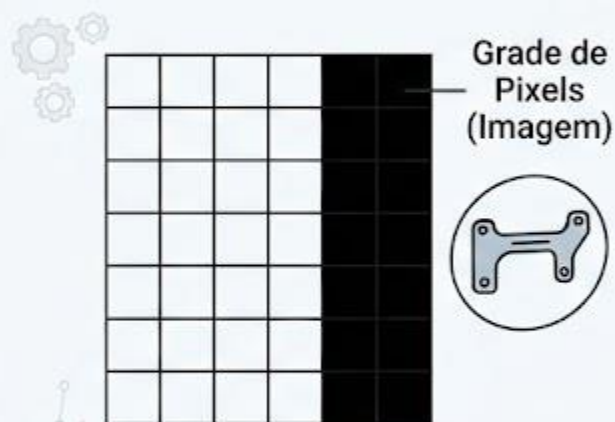
BREVE REVISÃO

DIAGRAMA DE EVOLUÇÃO DAS REDES NEURAIS (DO PERCEPTRON AO TRANSFORMERS)



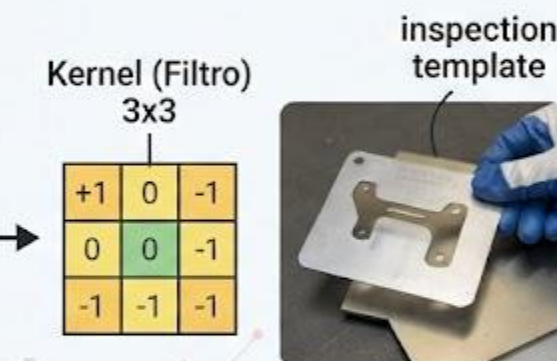
COMO FUNCIONA A CONVOLUÇÃO EM UMA CNN: ANALOGIA DO FILTRO DE INSPEÇÃO

1. A IMAGEM (A PEÇA A INSPECIONAR)



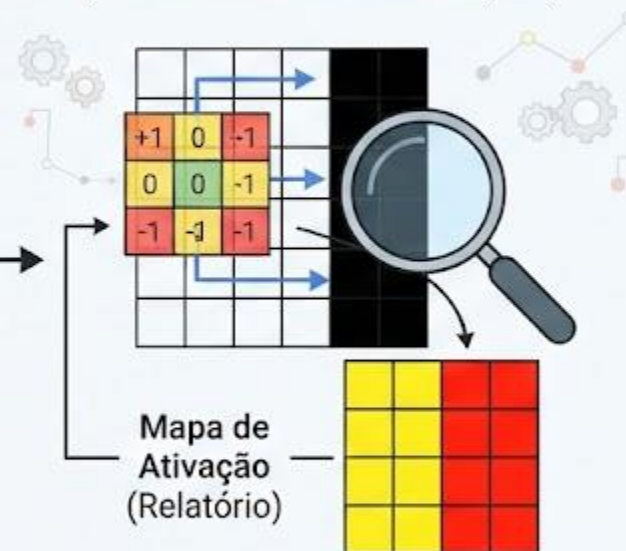
A imagem é apenas uma grade de números (pixels)

2. O KERNEL (O GABARITO DE INSPEÇÃO)



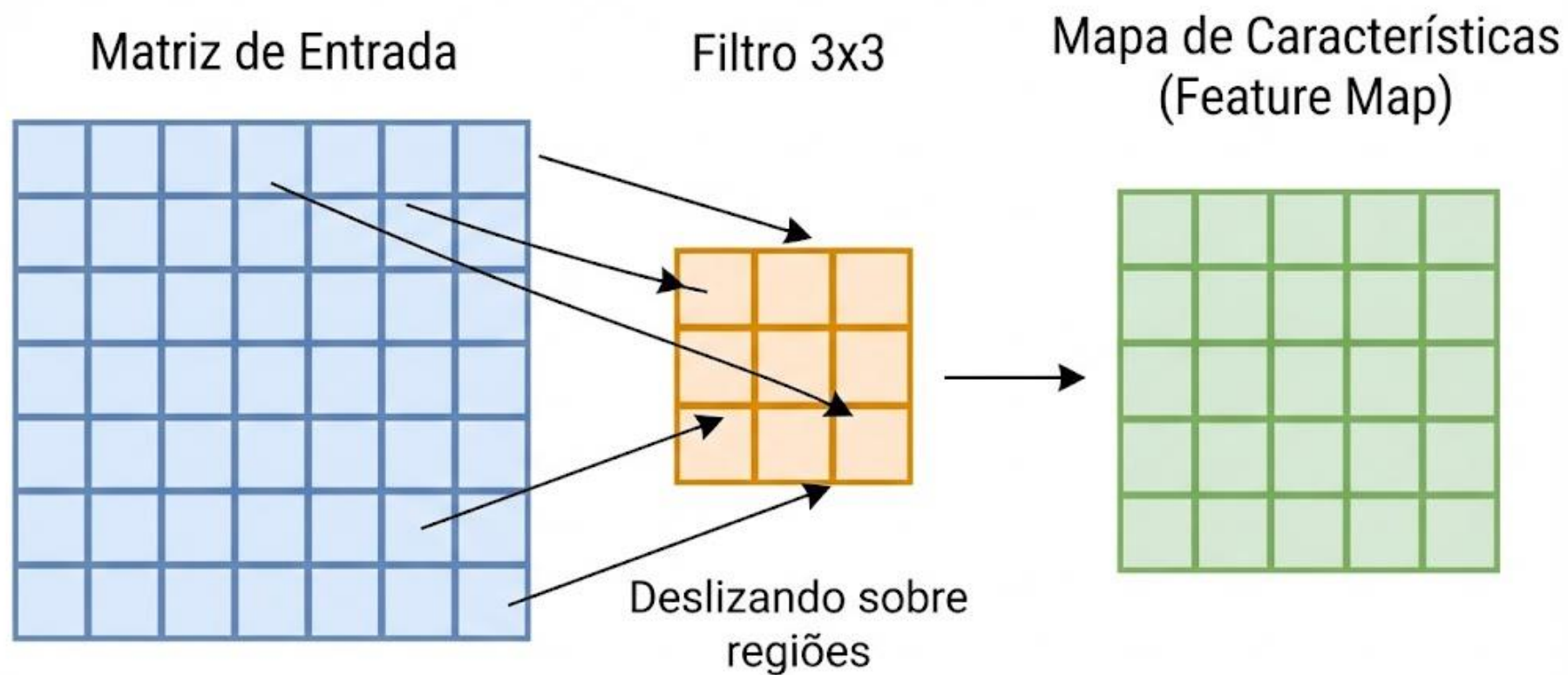
O kernel é um gabarito menor que procura por um padrão específico (ex: borda vertical)

3. O PROCESSO E MAPA DE ATIVAÇÃO (O RELATÓRIO DE INSPEÇÃO)



O kernel varre a imagem. O resultado da operação (soma) forma o Mapa de Ativação, destacando as características encontradas

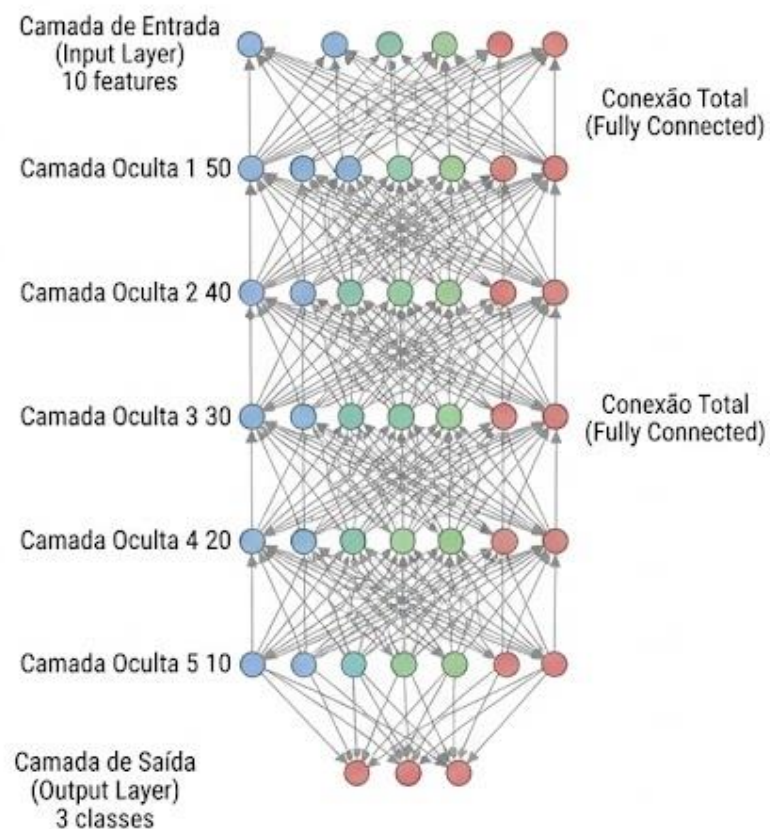
BREVE REVISÃO



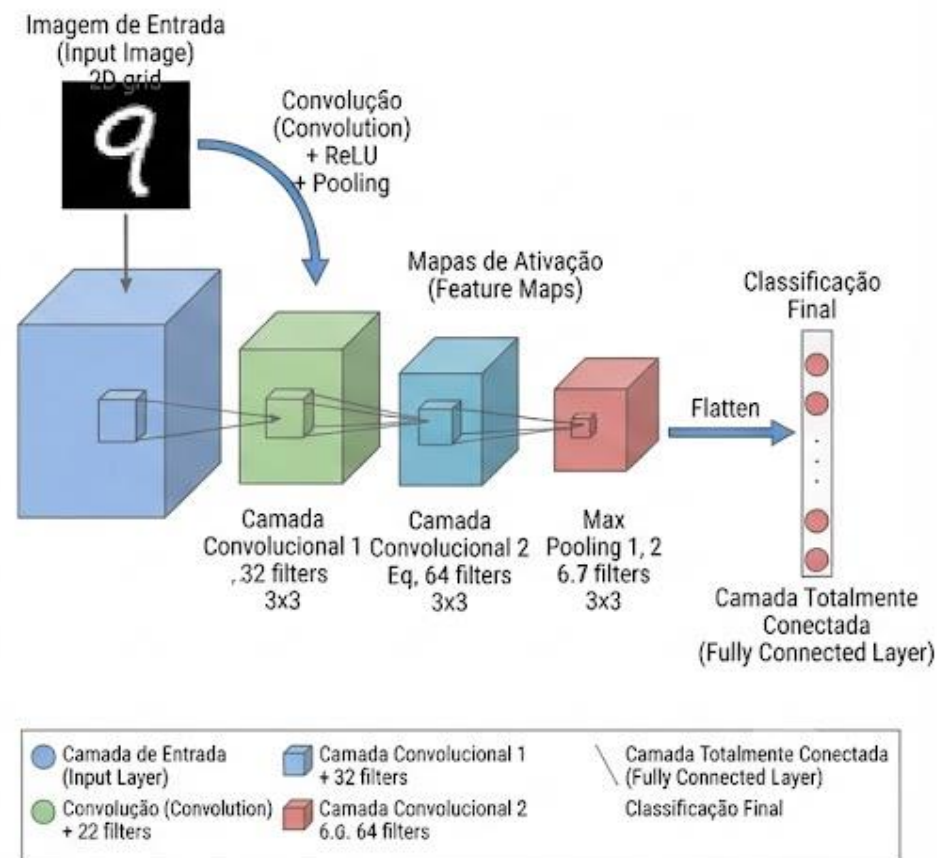
Entrada → Filtro → Característica extraída

BREVE REVISÃO

Deep Neural Network (DNN) Rede Neural Profunda



Convolutional Neural Network (CNN) Rede Neural Convolucional

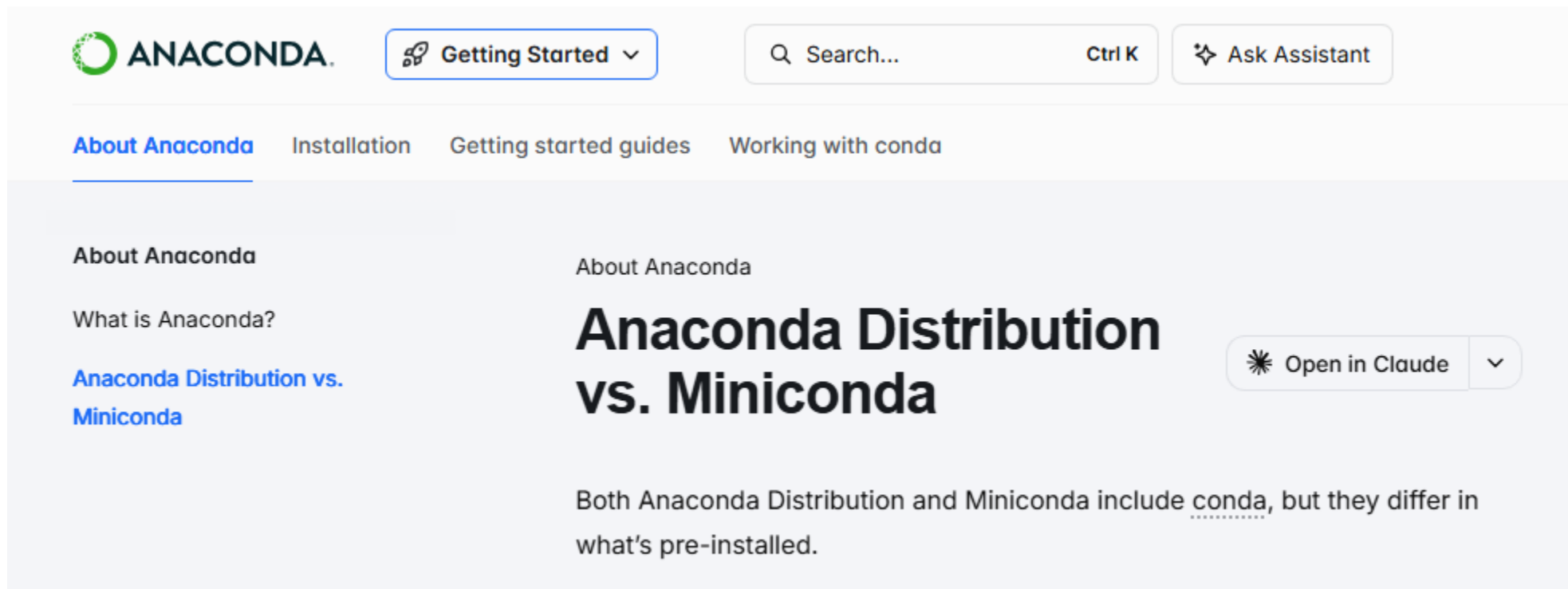


SUGESTÃO DE CONFIGURAÇÃO DO DEEP LEARNING NO KNIME

- ❑ Instalação do Miniconda (<https://www.anaconda.com/docs/getting-started/concepts/anaconda-or-miniconda>)
- ❑ Instalação das Seguintes Extensões no KNIME
 - Python / Conda / Keras

BREVE REVISÃO

SUGESTÃO DE CONFIGURAÇÃO DO DEEP LEARNING NO KNIME



The screenshot shows the Anaconda website's 'Getting started guides' section. The main article is titled 'Anaconda Distribution vs. Miniconda'. The article text states: 'Both Anaconda Distribution and Miniconda include conda, but they differ in what's pre-installed.' A button labeled 'Open in Claude' is visible next to the article title.

ANACONDA. Getting Started Search... Ctrl K Ask Assistant

About Anaconda Installation Getting started guides Working with conda

About Anaconda

What is Anaconda?

Anaconda Distribution vs. Miniconda

Anaconda Distribution vs. Miniconda

Open in Claude

Both Anaconda Distribution and Miniconda include conda, but they differ in what's pre-installed.

<https://www.anaconda.com/docs/getting-started/concepts/anaconda-or-miniconda>

BREVE REVISÃO

SUGESTÃO DE CONFIGURAÇÃO DO DEEP LEARNING NO KNIME

 Install

Available Software

Check the items that you wish to install.

Select All
Deselect All

Name	Version
KNIME & Extensions	
☐ KNIME Amazon Athena Connector	5.11.0.v202602010958
☐ KNIME Amazon Cloud Connectors	5.11.0.v202602201214
☐ KNIME Amazon Redshift Connector And Tools	5.10.0.v202601121533
☐ KNIME Amazon Redshift Driver (deprecated)	5.10.0.v202511251230
☐ KNIME Azure Cloud Connector	5.11.0.v202602211517
☐ KNIME Base C	
☐ KNIME Basic F	

3 items selected

Name	Version	Id
☒ KNIME Conda Integration	5.11.0.v202602201...	org.knime.features.conda.feature.group
☒ KNIME Deep Learning - Keras Integration	5.11.0.v202602010...	org.knime.features.dl.keras.feature.group
☒ KNIME Python Integration	5.11.0.v202602211...	org.knime.features.python3.scripting.fea...

SUGESTÃO DE CONFIGURAÇÃO DO DEEP LEARNING NO KNIME

- ❑ Configuração no KNIME (Preferences)
 - Conda
 - installation directory
 - Python DL
 - use special DL configuration : Keras
 - DL python env config: Conda
 - Keras: New environment

SUGESTÃO DE CONFIGURAÇÃO DO DEEP LEARNING NO KNIME

❑ Configuração no KNIME (Preferences)

- Python
 - env configuration = conda
 - conda env = py3_knime_dl

BREVE REVISÃO

Preferences (Filtered)

type filter text

- > General
- > Install/Update
- ▼ KNIME
 - Conda
 - Customization Profiles
 - Databases
 - JavaScript Views (legacy)
 - KNIME Explorer
 - > KNIME Modern UI
 - > KNIME classic user interf
 - Kerberos
 - Molecule Sketcher
 - Preferred Renderers
 - Python
 - > Python (legacy)
 - Python Deep Learning
 - > Table Backend
 - TensorFlow
 - > Workflow Coach

Conda

Conda Settings

Path to the Conda installation directory

C:\Users\jccte\miniconda3

Browse...

Conda version: 26.5.3

Pixi Environment Settings

☐ Store pixi environments temporarily

Base directory for pixi environments

Browse...

BREVE REVISÃO

Preferences (Filtered)

type filter text

- > General
- > Install/Update
- ▼ KNIME
 - Conda
 - Customization Profiles
 - Databases
 - JavaScript Views (legacy)
 - KNIME Explorer
 - > KNIME Modern UI
 - > KNIME classic user interf
 - Kerberos
 - Molecule Sketcher
 - Preferred Renderers
 - Python
 - > Python (legacy)
 - Python Deep Learning**
 - > Table Backend
 - TensorFlow
 - > Workflow Coach

Python Deep Learning

See [this guide](#) for details on how to setup the KNIME Deep Learning Integration.

☐ Use configuration from the "Python" preference page

☒ Use special Deep Learning configuration as defined below

Library used for the "DL Python" scripting nodes

☒ Keras ☐ TensorFlow 2

Deep Learning Python environment configuration

☒ Conda ☐ Manual

Please use the [Conda preference page](#) to configure the path to the Conda installation directory.
Using Conda at 'C:\Users\jccte\miniconda3'. Conda version: 26.5.3

Keras

Name of the Keras Conda environment

Python version: 3.6.15

TensorFlow 2

Name of the TensorFlow 2 Conda environment

Python version: 3.6.15

Library tensorflow is installed in version 1.14.0, required minimum version is 2.2.0 (inclusive), required maximum version is 3.0.0 (exclusive).

BREVE REVISÃO

Preferences (Filtered)

- > General
- > Install/Update
- ▼ KNIME
 - Conda
 - Customization Profiles
 - Databases
 - JavaScript Views (legacy)
 - KNIME Explorer
 - > KNIME Modern UI
 - > KNIME classic user interf
 - Kerberos
 - Molecule Sketcher
 - Preferred Renderers
 - Python**
 - > Python (legacy)
 - Python Deep Learning
 - > Table Backend
 - TensorFlow
 - > Workflow Coach

Python

See [this guide](#) for details on how to install Python for use with KNIME.

Python environment configuration

☐ Bundled ☒ Conda ☐ Manual

Please use the [Conda preference page](#) to configure the path to the Conda installation directory.

Using Conda at 'C:\Users\jccte\miniconda3'. Conda version: 26.5.3

Python 3

Name of the Python 3 Conda environment

Python version: 3.6.15



especialização

Aplicações IA/ML RIC: Redes Neurais Recorrentes (RNN)

Julio Tesolin

Open RAN Redes Abertas e Tecnologias Emergentes

REDES NEURAIS RECORRENTES - RNN

POR QUE REDES TEMPORAIS?

- ❑ Muitos fenômenos dependem do histórico anterior.
- ❑ Estado atual depende de eventos passados.
- ❑ Redes tradicionais ignoram relações temporais.
- ❑ RNNs aprendem padrões ao longo do tempo.

REDES NEURAIS RECORRENTES - RNN

DADOS POSSUEM MEMÓRIA

- ❑ Sequências representam eventos ordenados temporalmente.
- ❑ Ordem dos eventos influencia interpretação final.
- ❑ Histórico contém informações relevantes futuras.
- ❑ Modelos temporais exploram essa dependência.

REDES NEURAIS RECORRENTES - RNN

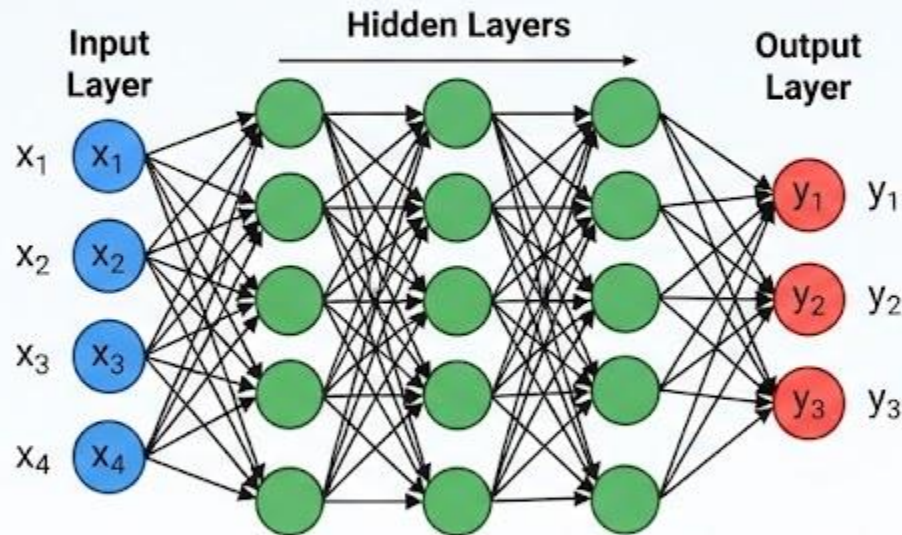
LIMITAÇÃO DAS REDES FEED FORWARD

- ❑ MLPs processam entradas independentes individualmente.
- ❑ DNNs não possuem memória interna temporal.
- ❑ Histórico precisa ser fornecido manualmente.
- ❑ Sequências longas tornam-se difíceis representar.

REDES NEURAIS RECORRENTES - RNN

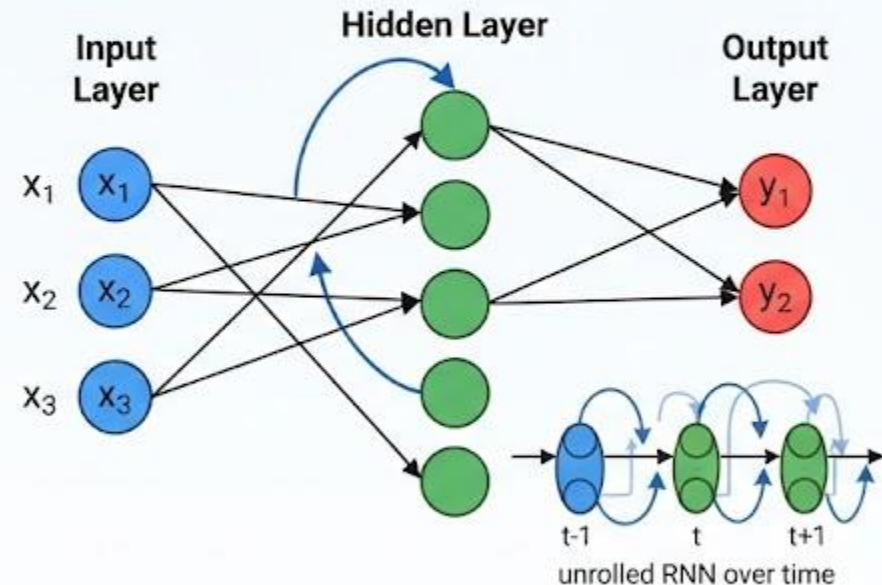
COMPARAÇÃO DE ARQUITETURAS DE REDES NEURAIS: DNN vs. RNN

DNN (Deep Neural Network - Rede Neural Profunda)



Arquitetura Feedforward, foco em mapeamento de entrada-saída (ex: Classificação de imagem).

RNN (Recurrent Neural Network - Rede Neural Recorrente)



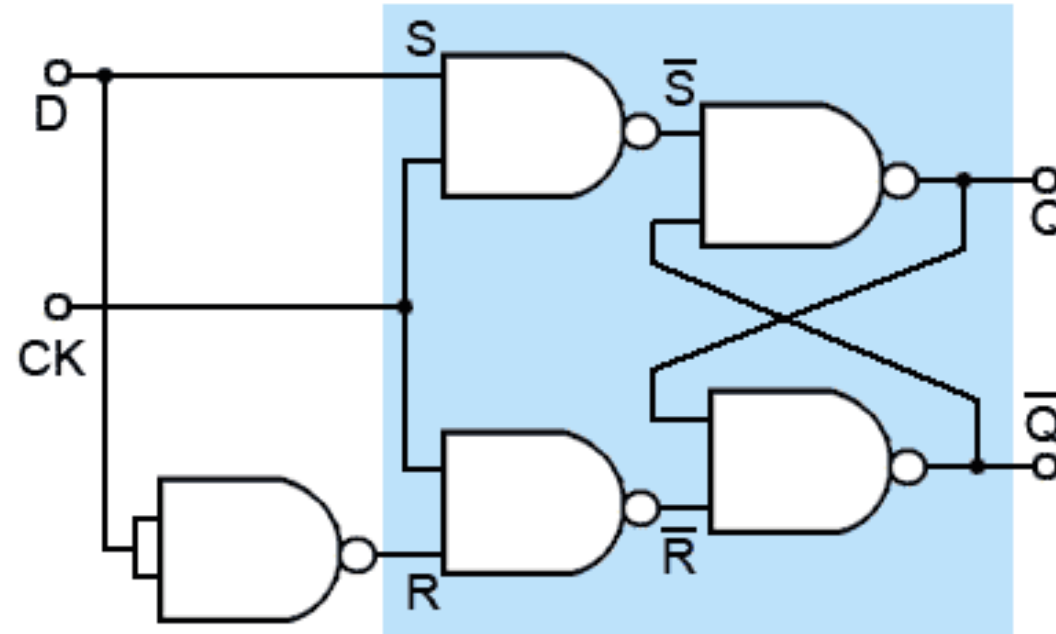
Arquitetura com ciclos, foco em processamento de sequências e dependências temporais (ex: Tradução, Processamento de linguagem natural).

REDES NEURAIS RECORRENTES - RNN

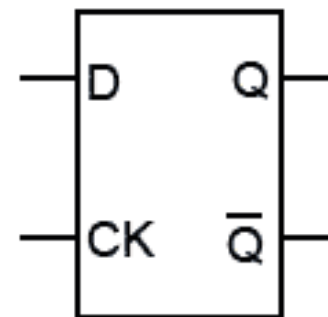
CONCEITO DE MEMÓRIA NEURAL

- ❑ RNN mantém informações de estados anteriores.
- ❑ Memória interna representa contexto temporal.
- ❑ Estado oculto acumula informações relevantes.
- ❑ Rede utiliza passado durante previsões.

REDES NEURAIS RECORRENTES - RNN



Inputs		Outputs	
CK	D	Q	$\overline{Q}$
0	X	No change	
1	0	0	1
1	1	1	0



FLIP FLOP D

REDES NEURAIS RECORRENTES - RNN

RESUMO: MODELAGEM ESTATÍSTICA ARIMA (AutoRegressive Integrated Moving Average)

O QUE É ARIMA? (CONTEXTO ESTATÍSTICO)

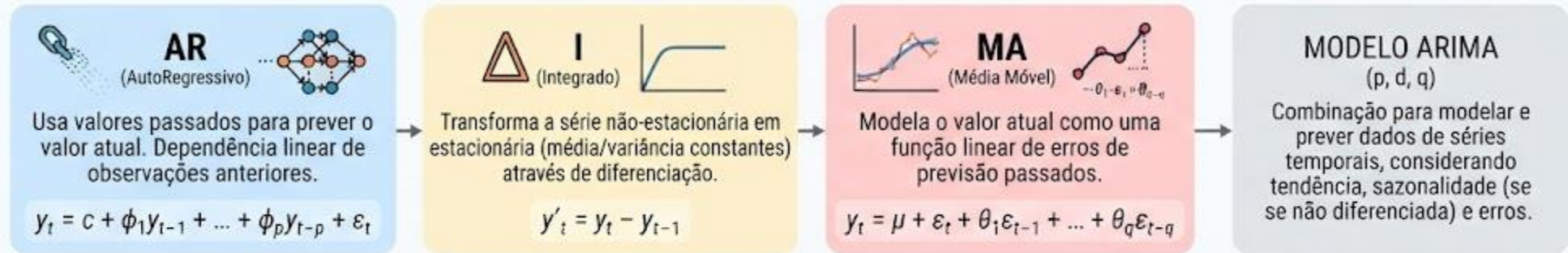
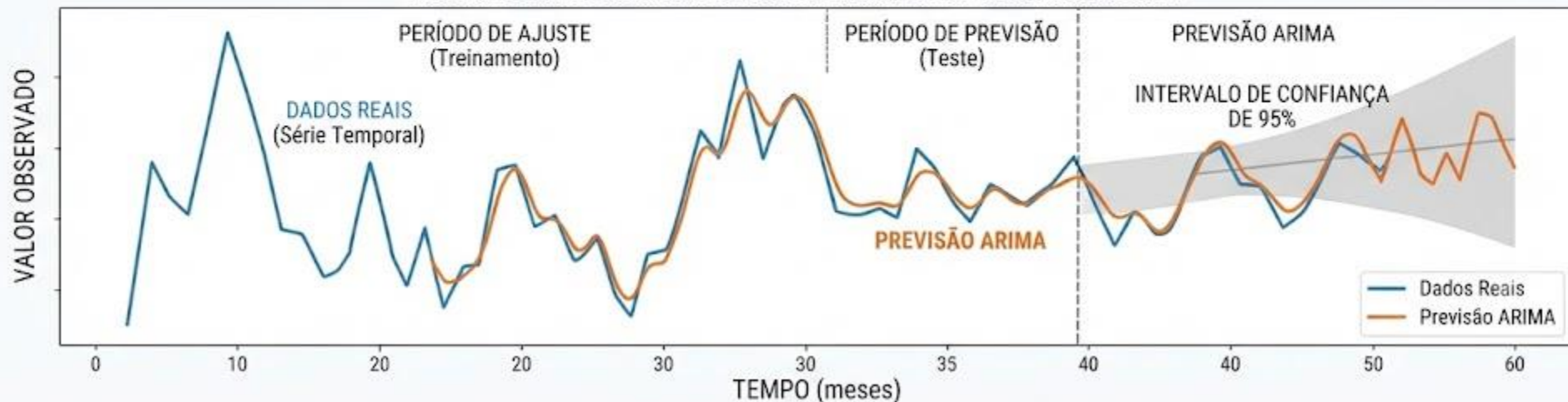


FIGURA: CURVA TEMPORAL SOBREPOSTA COM ARIMA

ANÁLISE COMPARATIVA: DADOS REAIS VS. PREVISÃO ARIMA



REDES NEURAIS RECORRENTES - RNN

FUNCIONAMENTO BÁSICO DA RNN

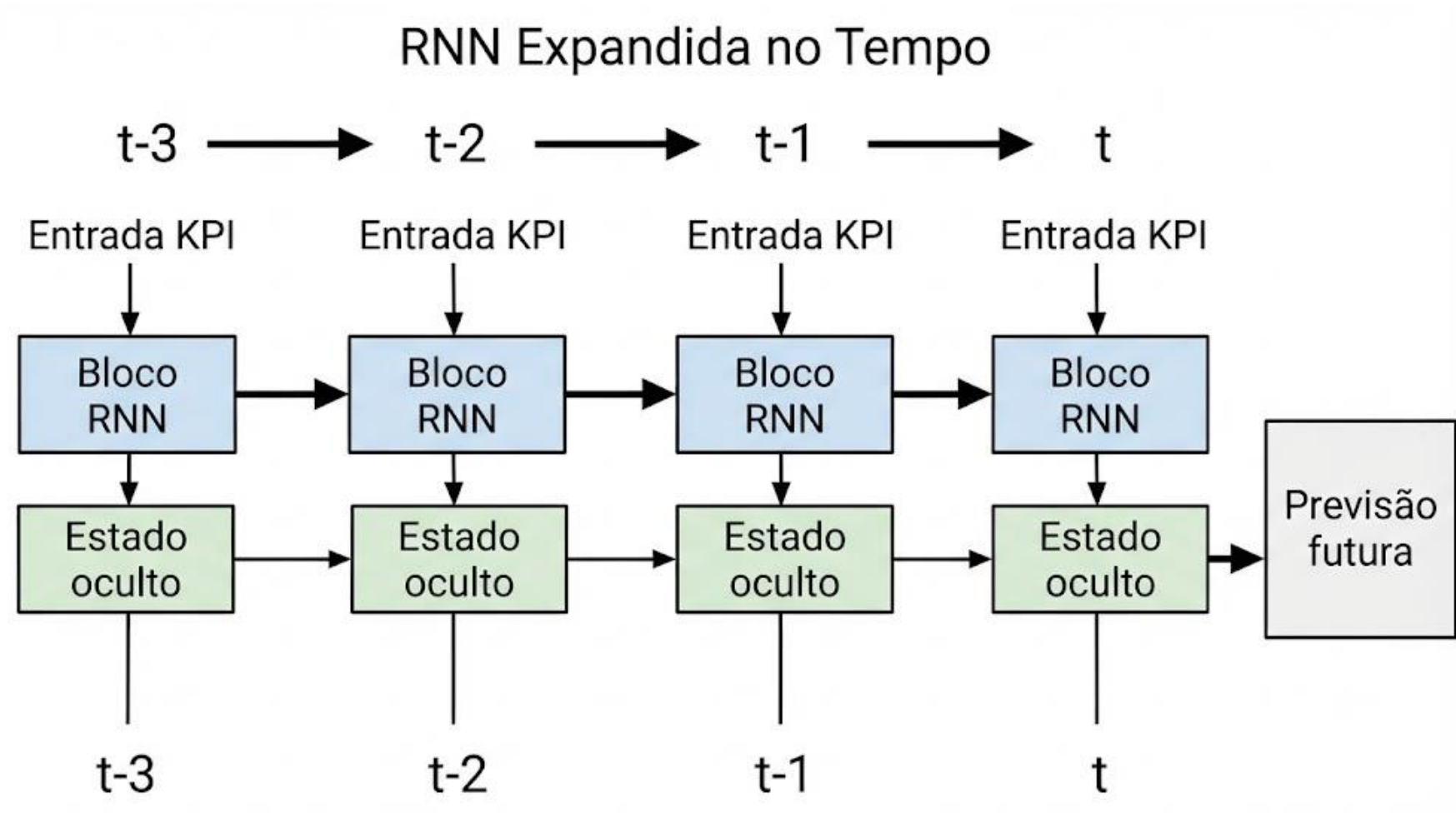
- ❑ Entrada atual combina informações anteriores.
- ❑ Estado oculto é atualizado continuamente.
- ❑ Mesmos pesos processam diferentes instantes.
- ❑ Saída depende contexto acumulado.

REDES NEURAIS RECORRENTES - RNN

ESTADO OCULTO: A MEMÓRIA DA REDE

- ❑ Estado oculto armazena contexto temporal.
- ❑ Informações antigas influenciam novas previsões.
- ❑ Atualização ocorre a cada instante.
- ❑ Memória representa comportamento da sequência.

REDES NEURAIS RECORRENTES - RNN



REDES NEURAIS RECORRENTES - RNN

PROBLEMA DO GRADIENTE DESVANECENTE

- ❑ Treinamento depende da propagação do erro.
- ❑ Gradientes podem diminuir progressivamente.
- ❑ Informações antigas tornam-se difíceis aprender.
- ❑ Problema limita capacidade temporal.

REDES NEURAIS RECORRENTES - RNN

LIMITAÇÕES DAS RNNS TRADICIONAIS

- ❑ Memória limitada para sequências longas.
- ❑ Informações antigas podem ser perdidas.
- ❑ Treinamento apresenta dificuldades práticas.
- ❑ LSTM surge para resolver limitações.

REDES NEURAIS RECORRENTES - RNN

LONG SHORT-TERM MEMORY (LSTM)

- ❑ LSTM adiciona controle sobre informações armazenadas.
- ❑ Memória decide conteúdos importantes a manter.
- ❑ Portas (**Gates**) controlam fluxo de informações.
- ❑ Dependências longas tornam-se aprendíveis.

REDES NEURAIS RECORRENTES - RNN

DIFERENÇA ENTRE RNN SIMPLES E LSTM: UMA METÁFORA SIMPLES



REDES NEURAIS RECORRENTES - RNN

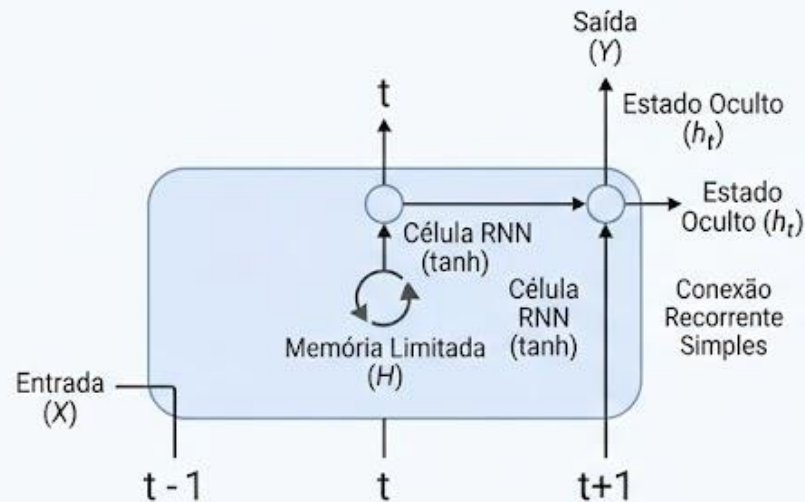
PORTAS (GATES) DA LSTM

- ❑ **Forget Gate:** remove informações desnecessárias.
- ❑ **Input Gate:** adiciona novos conhecimentos relevantes.
- ❑ **Output Gate:** controla informação utilizada.
- ❑ **Memória** torna-se mais eficiente temporalmente.

REDES NEURAIS RECORRENTES - RNN

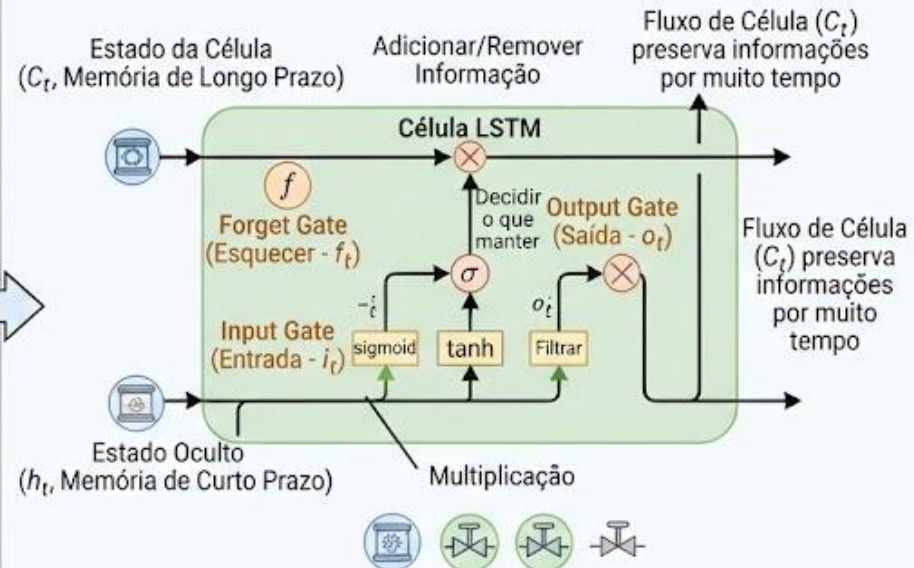
DIFERENÇA ENTRE RNN SIMPLES E LSTM: MEMÓRIA E GATES

RNN SIMPLES (Rede Neural Recorrente)



PRINCÍPIO: Estado oculto atual é uma função apenas da entrada atual e do estado anterior. Sofre de "Gradiente Desvanecente" (Esquece informações longas).

LSTM (Long Short-Term Memory)



PRINCÍPIO: Mecanismos de "gates" regulam o fluxo de informação, permitindo adicionar ou remover dados da memória de longo prazo. Resolve o "Gradiente Desvanecente".

REDES NEURAIS RECORRENTES - RNN

APRENDIZADO TEMPORAL HIERÁRQUICO

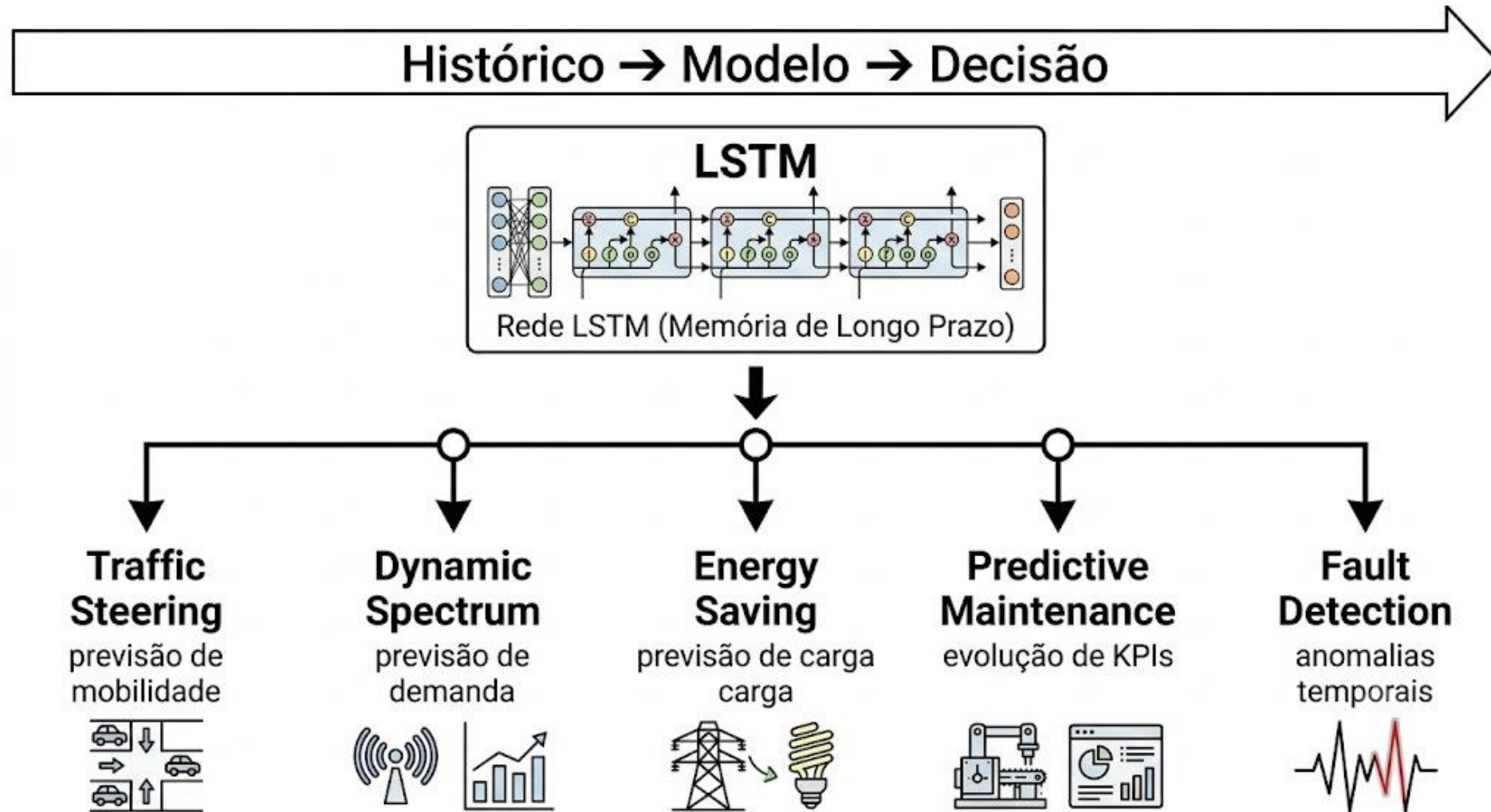
- ❑ LSTM identifica tendências ao longo tempo.
- ❑ Padrões repetitivos são automaticamente aprendidos.
- ❑ Eventos anteriores influenciam previsões futuras.
- ❑ Modelo captura comportamentos complexos.

REDES NEURAIS RECORRENTES - RNN

RNN / LSTM EM REDES MÓVEIS

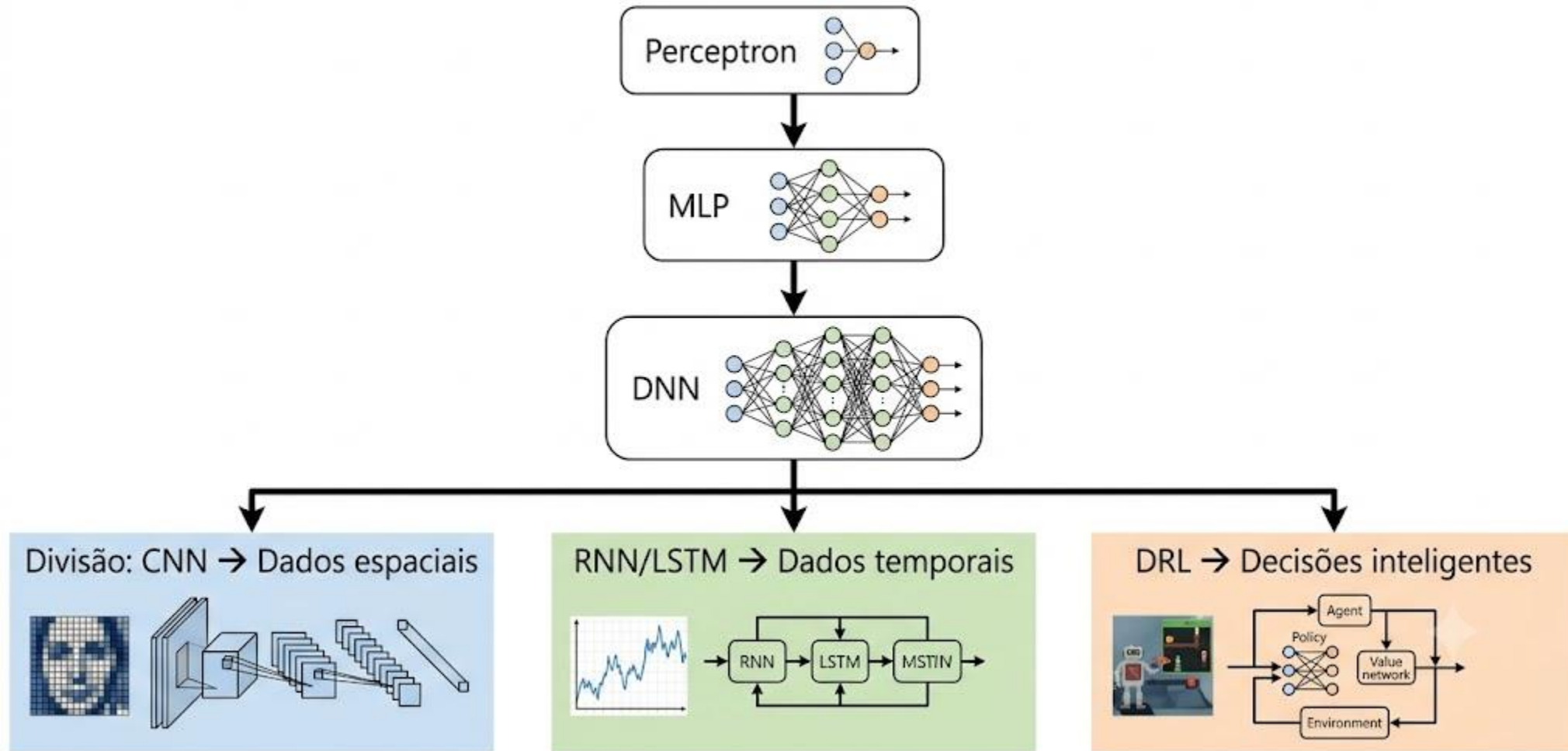
- ❑ Histórico de KPIs alimenta modelos temporais.
- ❑ Previsões auxiliam decisões automatizadas.
- ❑ Tendências operacionais são identificadas antecipadamente.
- ❑ Aplicações melhoram eficiência da rede.

REDES NEURAIS RECORRENTES - RNN



REDES NEURAIS RECORRENTES - RNN

Redes neurais adaptadas ao tipo de problema



PRATICA KNIME

MULTIVARIATE TIME SERIES ANALYSIS WITH AN RNN - TRAINING

https://hub.knime.com/kathrin/spaces/Multivariate%20Times%20Series%20with%20RNN/Multivariate_Time_Series_RNN_Keras_Training~B45XEOAuWeQBzO9b/current-state



especialização

Aplicações IA/ML RIC: Aprendizado por Representação

Julio Tesolin

Open RAN Redes Abertas e Tecnologias Emergentes

APRENDIZADO POR REPRESENTAÇÃO

O DESAFIO DAS FALHAS DESCONHECIDAS

- ❑ Redes móveis geram grandes volumes dados.
- ❑ Falhas conhecidas representam pequena parcela.
- ❑ Novos comportamentos anormais surgem continuamente.
- ❑ Modelos tradicionais possuem limitações práticas.

APRENDIZADO POR REPRESENTAÇÃO

LIMITAÇÃO DO APRENDIZADO SUPERVISIONADO

- ❑ Modelos dependem de exemplos previamente rotulados.
- ❑ Falhas raras possuem poucos registros históricos.
- ❑ Classes desconhecidas dificultam classificação automática.
- ❑ Rotulagem manual exige esforço operacional elevado.

APRENDIZADO POR REPRESENTAÇÃO

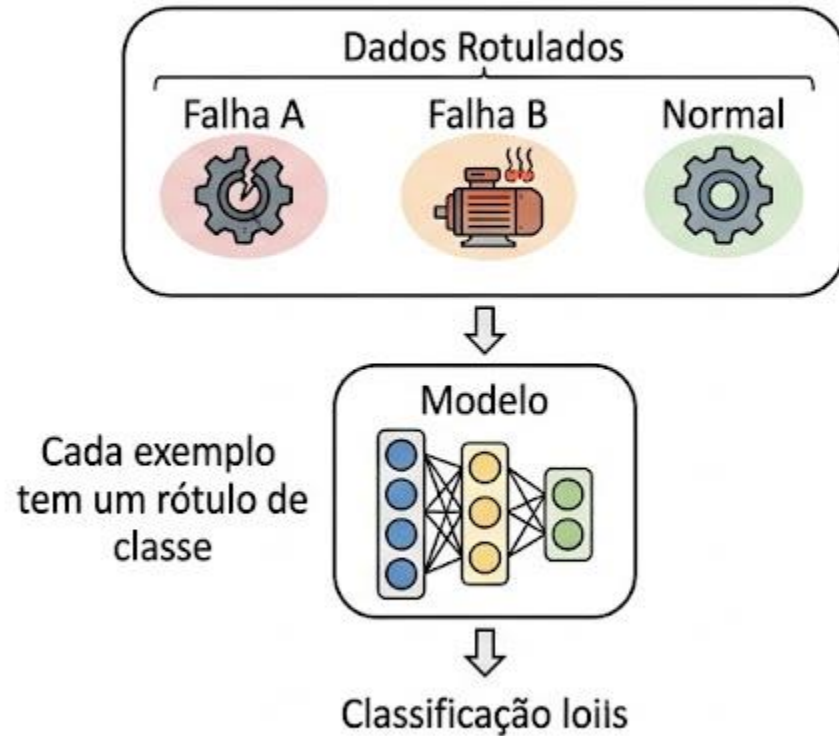
APRENDER REPRESENTAÇÕES DOS DADOS

- ❑ Redes neurais podem extrair características automaticamente.
- ❑ Dados complexos possuem padrões ocultos relevantes.
- ❑ Representações compactas facilitam análise inteligente.
- ❑ Autoencoders aprendem estruturas sem rótulos.

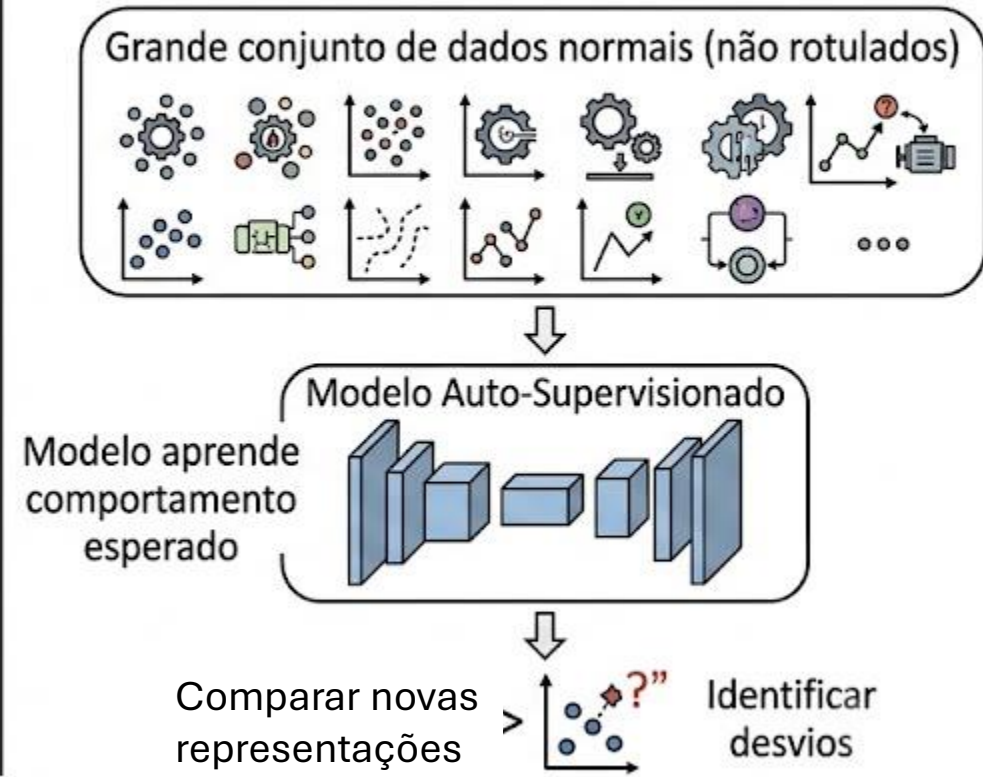
APRENDIZADO POR REPRESENTAÇÃO

Abordagens de Aprendizado para Detecção de Falhas

Lado Esquerdo: Aprendizado Supervisionado



Lado Direito: Aprendizado por Representação



“Conhecer classes” × “Conhecer padrões”

APRENDIZADO POR REPRESENTAÇÃO

O CONCEITO DO AUTOENCODER

- ❑ Autoencoder aprende reconstruir seus próprios dados.
- ❑ Rede comprime informações importantes internamente.
- ❑ Representação reduz dimensionalidade dos dados.
- ❑ Treinamento ocorre sem classes explícitas.

APRENDIZADO POR REPRESENTAÇÃO

ENCODER: EXTRAINDO CARACTERÍSTICAS

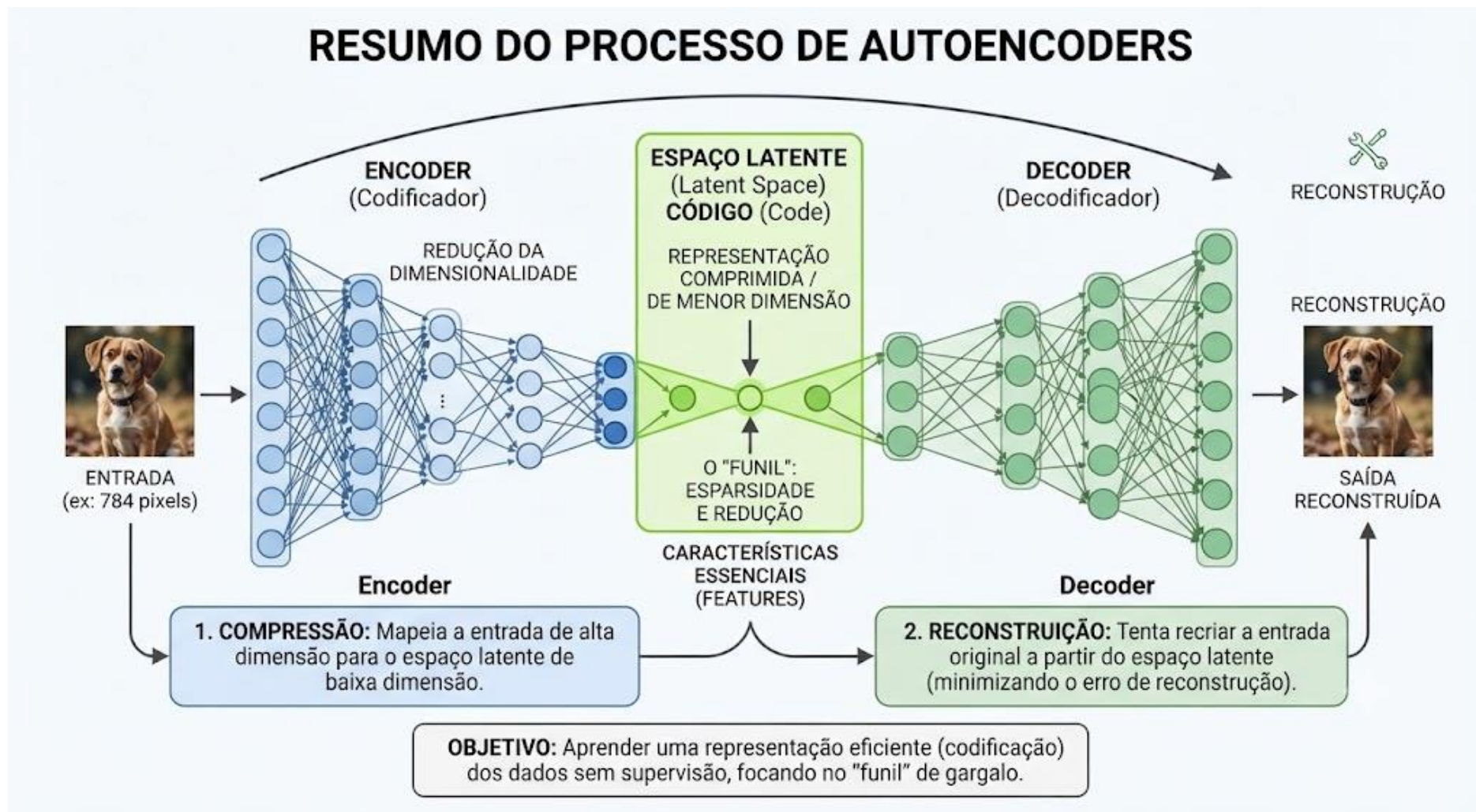
- ❑ Encoder transforma dados em representação menor.
- ❑ Características importantes permanecem preservadas.
- ❑ Informações redundantes são removidas progressivamente.
- ❑ Espaço interno representa comportamento aprendido.

APRENDIZADO POR REPRESENTAÇÃO

DECODER: RECONSTRUINDO INFORMAÇÕES

- ❑ Decoder recupera dados originais comprimidos.
- ❑ Qualidade depende representação aprendida.
- ❑ Reconstrução próxima indica padrão conhecido.
- ❑ Reconstrução ruim indica comportamento diferente.

APRENDIZADO POR REPRESENTAÇÃO



APRENDIZADO POR REPRESENTAÇÃO

ERRO DE RECONSTRUÇÃO

- ❑ Diferença mede qualidade da reconstrução.
- ❑ Pequeno erro indica padrão conhecido.
- ❑ Grande erro indica possível anomalia.
- ❑ Métrica permite monitoramento contínuo.

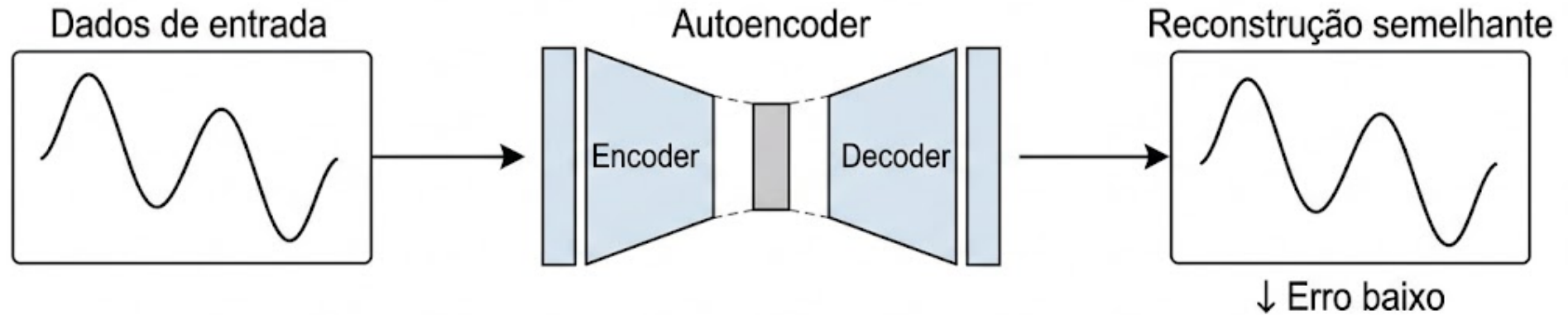
APRENDIZADO POR REPRESENTAÇÃO

DETECÇÃO DE ANOMALIAS AUTOMÁTICA

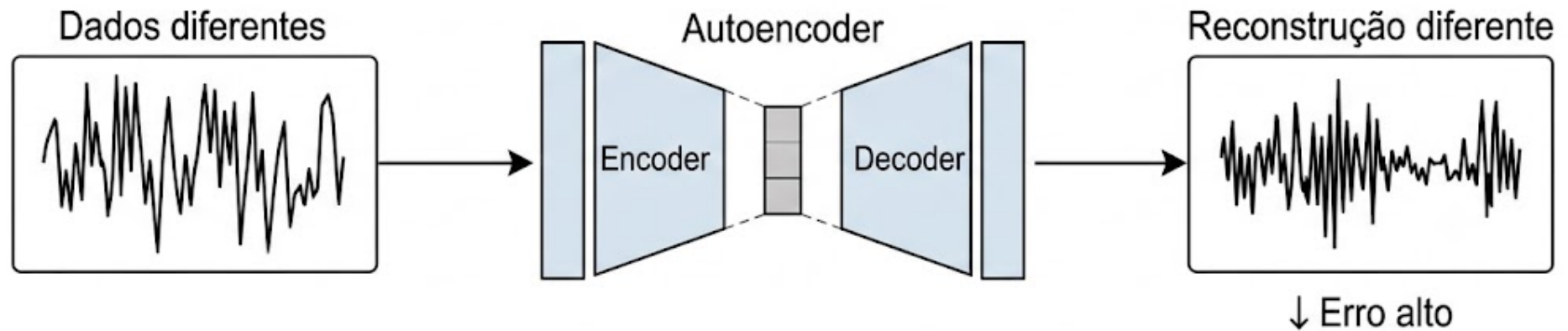
- ❑ Modelo aprende comportamento normal da rede.
- ❑ Novos dados são comparados automaticamente.
- ❑ Desvios recebem maior pontuação anormal.
- ❑ Operadores recebem alertas antecipados.

APRENDIZADO POR REPRESENTAÇÃO

Parte Superior: Comportamento Normal



Parte Inferior: Comportamento Anômalo



APRENDIZADO POR REPRESENTAÇÃO

LIMITAÇÕES DO AUTOENCODER TRADICIONAL

- ❑ Representação aprendida pode ser limitada.
- ❑ Generalização depende dos dados disponíveis.
- ❑ Espaço latente possui interpretação limitada.
- ❑ Modelos probabilísticos ampliam capacidade.

APRENDIZADO POR REPRESENTAÇÃO

VARIATIONAL AUTOENCODER (VAE)

- ❑ VAE amplia conceito do Autoencoder.
- ❑ Representação possui natureza probabilística.
- ❑ Modelo aprende distribuição dos dados.
- ❑ Incertezas tornam-se parte da análise.

APRENDIZADO POR REPRESENTAÇÃO

ESPAÇO LATENTE PROBABILÍSTICO

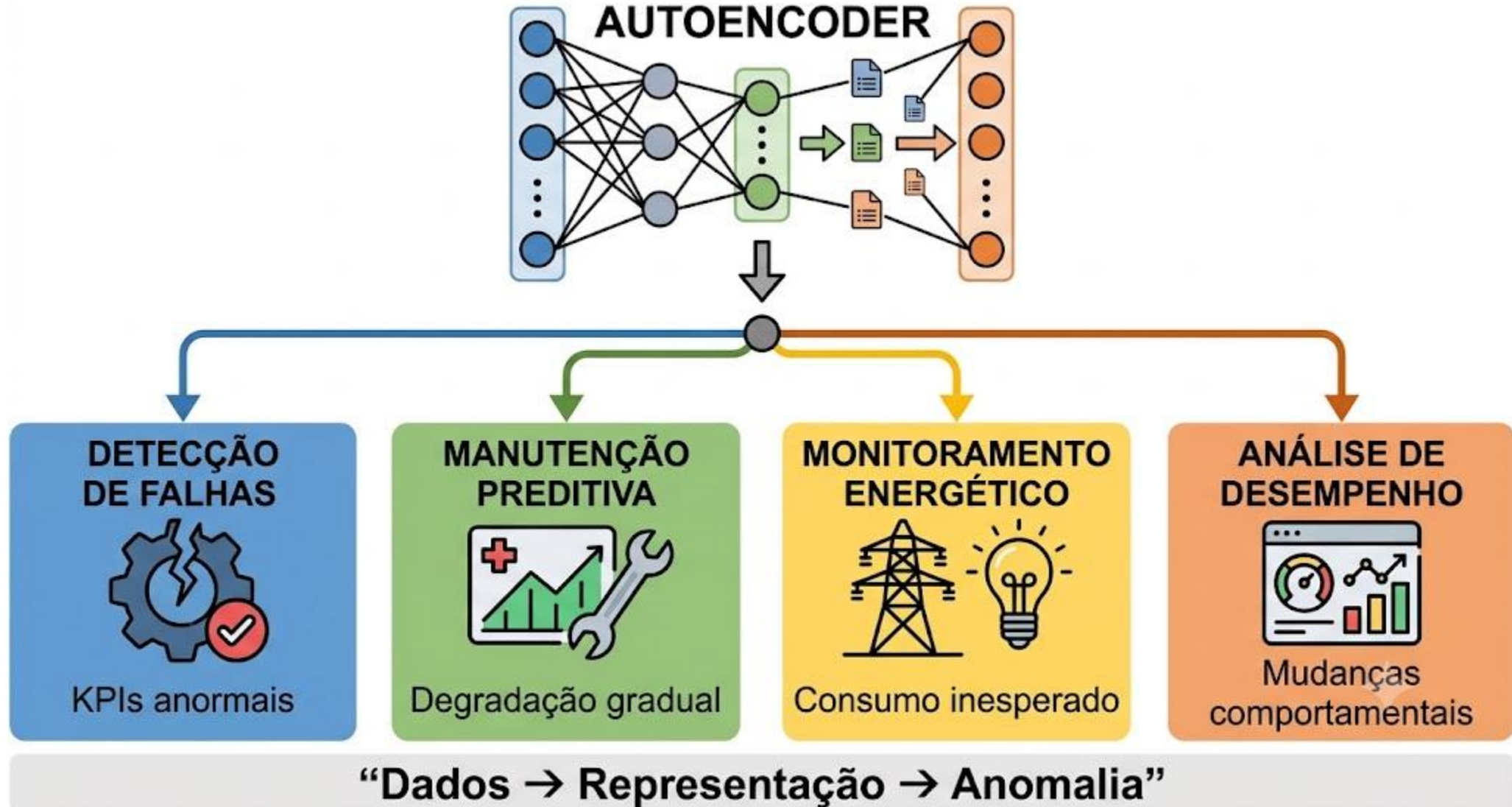
- ❑ VAE organiza representações em regiões.
- ❑ Dados semelhantes permanecem próximos.
- ❑ Transições tornam-se mais suaves.
- ❑ Novos padrões podem ser identificados.

APRENDIZADO POR REPRESENTAÇÃO

AUTOENCODERS NO OPEN RAN

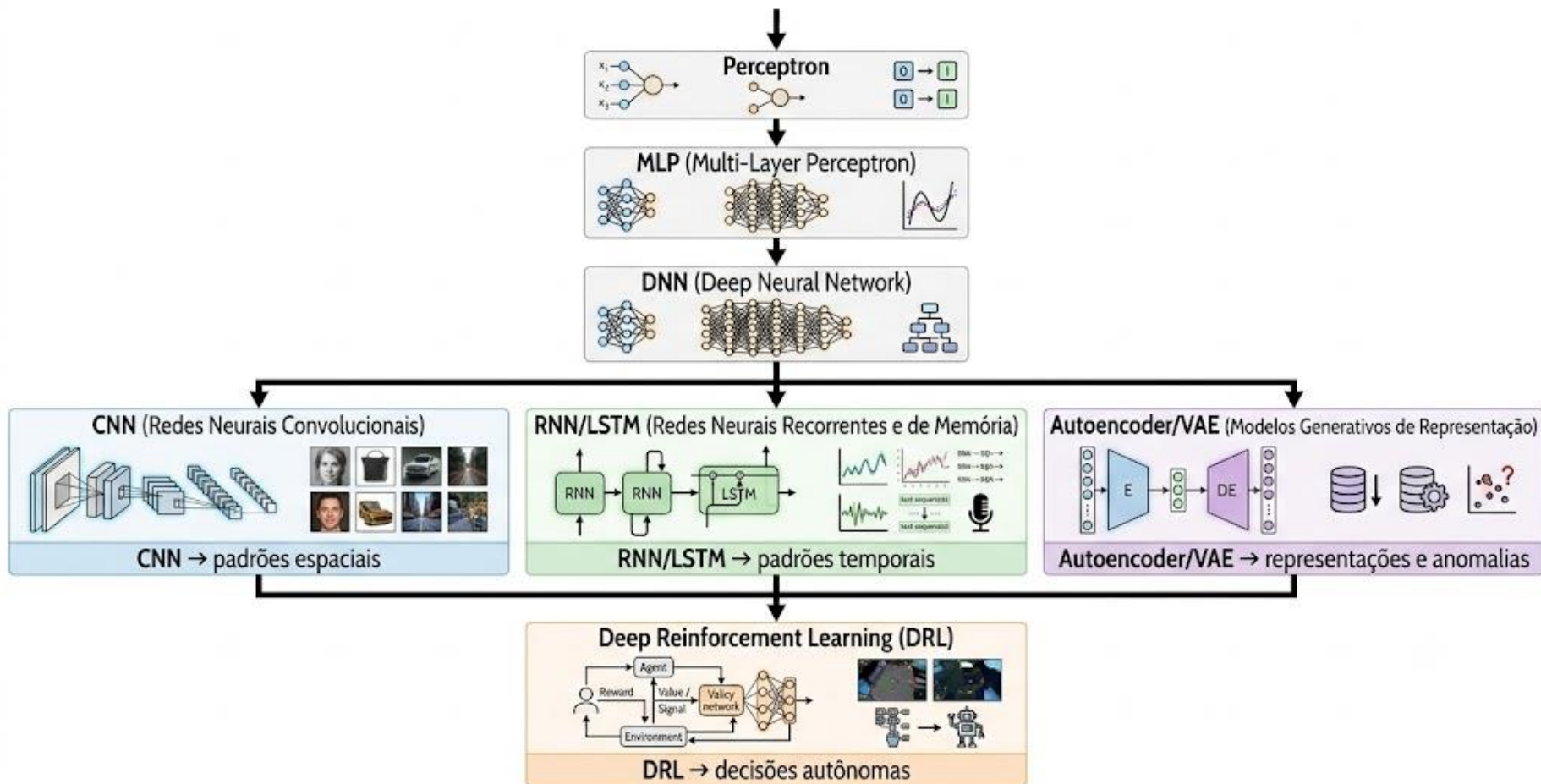
- ❑ KPIs alimentam modelos de representação.
- ❑ Padrões normais são aprendidos automaticamente.
- ❑ Anomalias recebem indicadores quantitativos.
- ❑ Decisões operacionais tornam-se inteligentes.

APRENDIZADO POR REPRESENTAÇÃO



APRENDIZADO POR REPRESENTAÇÃO

Linha Evolutiva das Redes Neurais Profundas



PRATICA KNIME

Autoencoder for anomaly detection

<https://hub.knime.com/knime/spaces/Education/Educational%20Videos%20Demo%20Workflows/Autoencoder%20for%20anomaly%20detection~6fhhHxcGwN2LZiGA/current-state>



especialização

Aplicações IA/ML RIC: Aprendizado por Reforço

Julio Tesolin

Open RAN Redes Abertas e Tecnologias Emergentes

APRENDIZADO POR REFORÇO

DA PREVISÃO À DECISÃO

- ❑ Modelos anteriores aprendiam padrões históricos.
- ❑ Redes neurais realizavam previsões específicas.
- ❑ Sistemas autônomos precisam escolher ações.
- ❑ Aprendizado por reforço aprende decisões inteligentes.

APRENDIZADO POR REFORÇO

LIMITE DOS MODELOS TRADICIONAIS

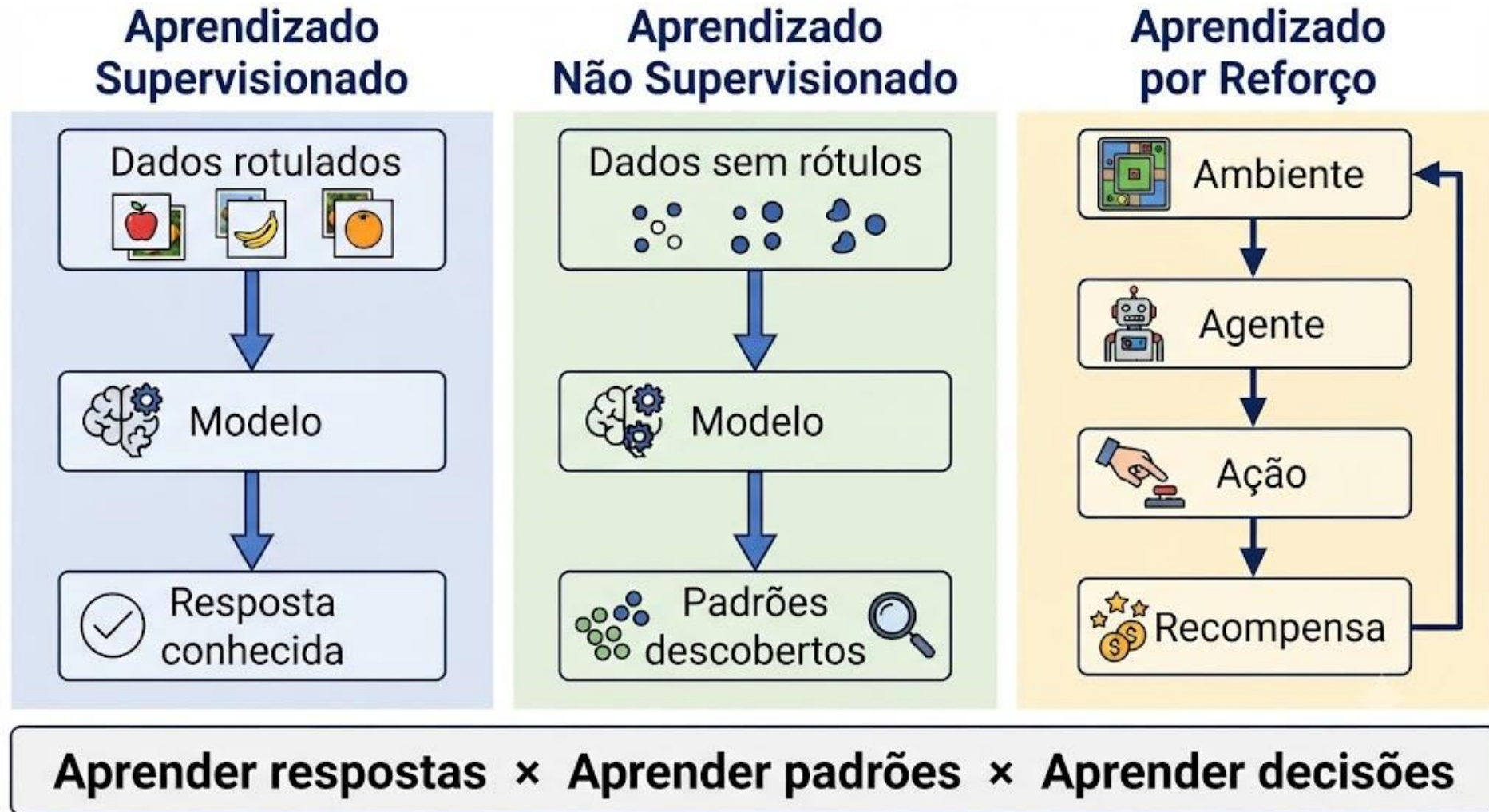
- ❑ Aprendizado supervisionado depende exemplos rotulados.
- ❑ Modelos preveem resultados previamente definidos.
- ❑ Decisões operacionais exigem ações adaptativas.
- ❑ Ambientes dinâmicos exigem aprendizado contínuo.

APRENDIZADO POR REFORÇO

FILOSOFIA DO APRENDIZADO POR REFORÇO

- ❑ Agentes aprendem através da interação.
- ❑ Ações geram consequências no ambiente.
- ❑ Recompensas orientam melhorias futuras.
- ❑ Experiência substitui exemplos supervisionados.

APRENDIZADO POR REFORÇO



APRENDIZADO POR REFORÇO

APRENDIZADO PELA EXPERIÊNCIA

- ❑ Experiência substitui conhecimento prévio completo.
- ❑ Agente explora diferentes possibilidades operacionais.
- ❑ Resultados anteriores influenciam decisões futuras.
- ❑ Aprendizado melhora continuamente com interação.

APRENDIZADO POR REFORÇO

AGENTE

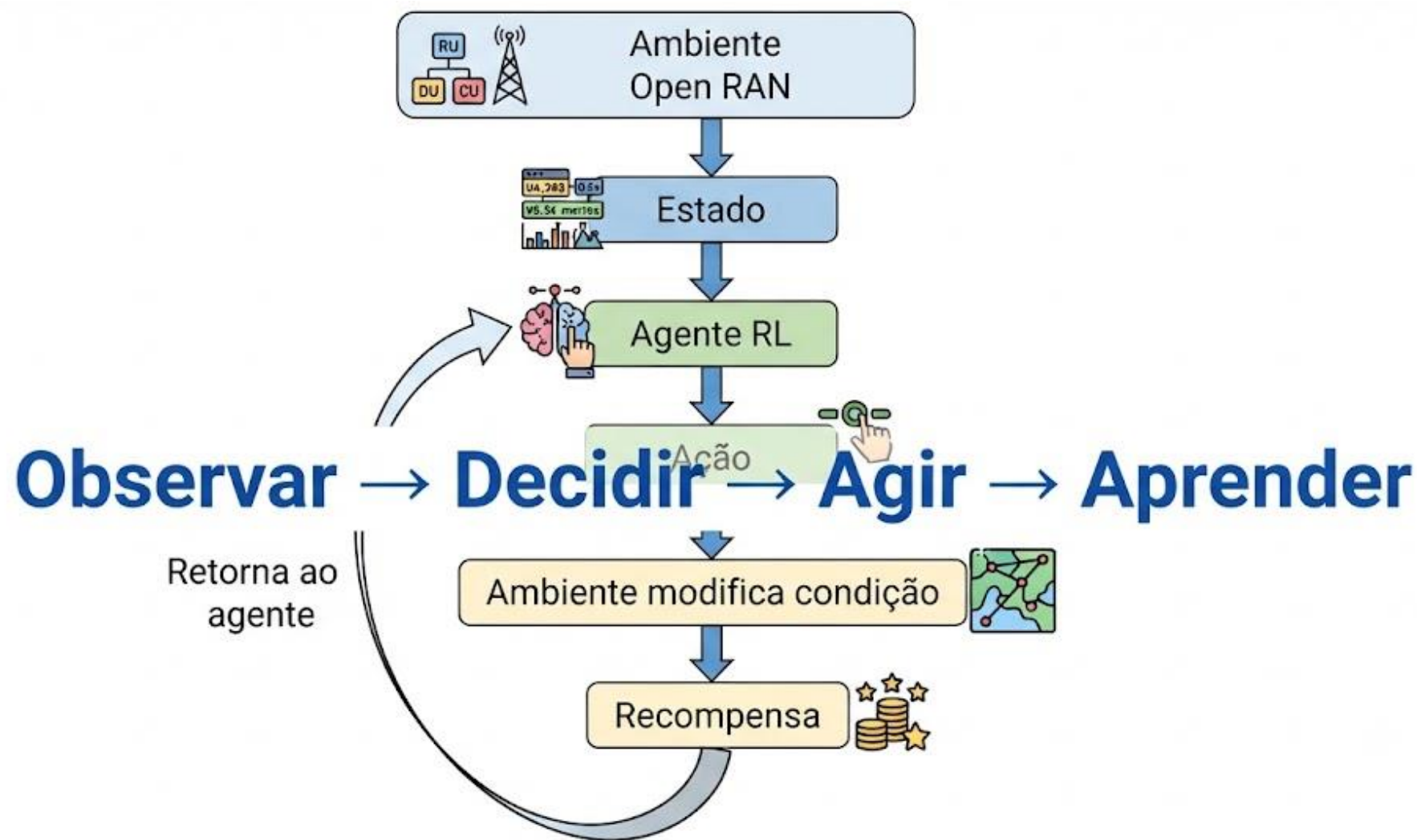
- ❑ Agente representa entidade decisória.
- ❑ Observa ambiente antes das ações.
- ❑ Escolhe comportamento conforme objetivo.
- ❑ Aprende política operacional progressivamente.

APRENDIZADO POR REFORÇO

AMBIENTE, ESTADO E OBSERVAÇÃO

- ❑ Ambiente representa sistema controlado.
- ❑ Estado descreve condição atual observada.
- ❑ Informações definem contexto decisório.
- ❑ Qualidade depende representação adequada.

APRENDIZADO POR REFORÇO



APRENDIZADO POR REFORÇO

AÇÕES E CONSEQUÊNCIAS FUTURAS

- ❑ Cada ação modifica ambiente operacional.
- ❑ Resultados podem aparecer posteriormente.
- ❑ Decisões imediatas possuem impactos futuros.
- ❑ RL considera consequências acumuladas.

APRENDIZADO POR REFORÇO

RECOMPENSA

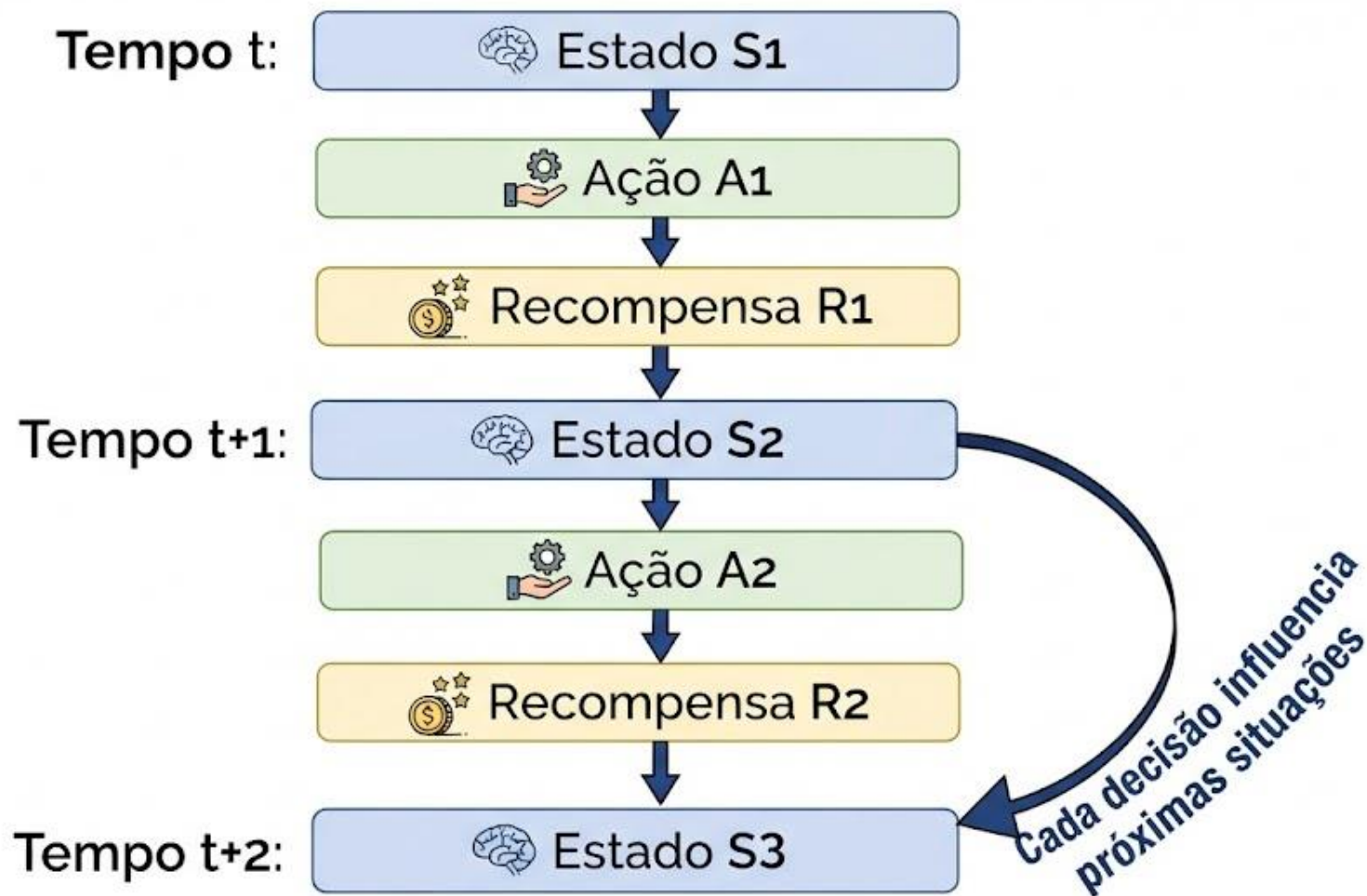
- ❑ Recompensa avalia qualidade das ações.
- ❑ Valores positivos incentivam bons comportamentos.
- ❑ Penalidades evitam decisões inadequadas.
- ❑ Objetivo maximiza retorno acumulado.

APRENDIZADO POR REFORÇO

POLÍTICA DE DECISÃO

- ❑ Política define comportamento do agente.
- ❑ Relaciona estados com ações possíveis.
- ❑ Estratégias melhores são gradualmente aprendidas.
- ❑ Objetivo encontra decisões eficientes.

APRENDIZADO POR REFORÇO

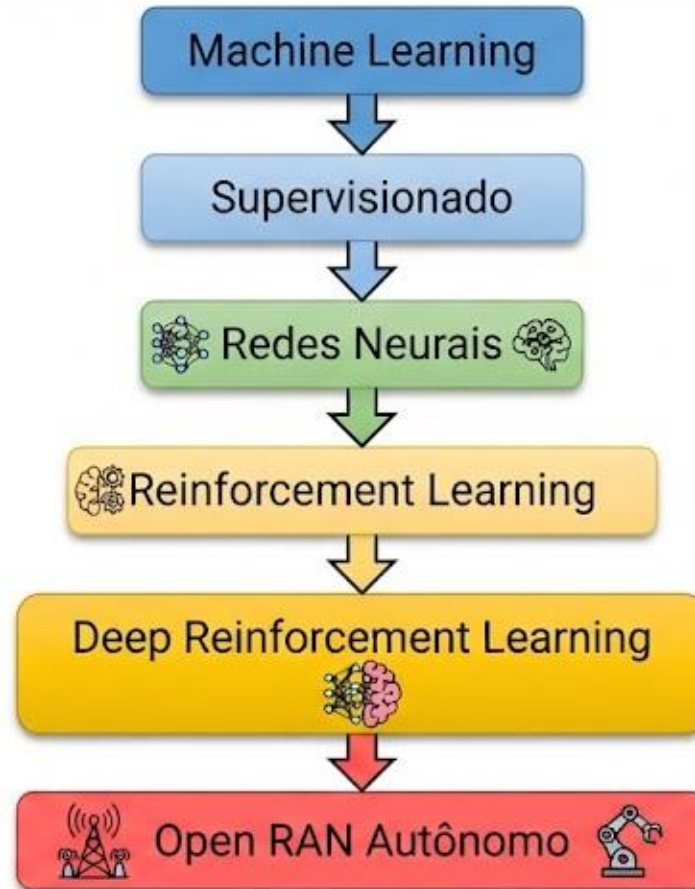


APRENDIZADO POR REFORÇO

RL APLICADO AO OPEN RAN

- ❑ Redes móveis possuem decisões dinâmicas.
- ❑ KPIs representam estados operacionais.
- ❑ Ações controlam recursos da rede.
- ❑ Recompensas representam objetivos técnicos.

APRENDIZADO POR REFORÇO



Aprender padrões → Aprender decisões



especialização

Aplicações IA/ML RIC: Markov Decision Process (MDP)

Julio Tesolin

Open RAN Redes Abertas e Tecnologias Emergentes

MARKOV DECISION PROCESS - MDP

DA INTERAÇÃO AO MODELO FORMAL

- ❑ Agente aprende interagindo com ambiente.
- ❑ Decisões dependem das condições atuais.
- ❑ Resultados influenciam estratégias futuras.
- ❑ MDP representa problemas sequenciais formalmente.

MARKOV DECISION PROCESS - MDP

PROCESSO DE DECISÃO DE MARKOV

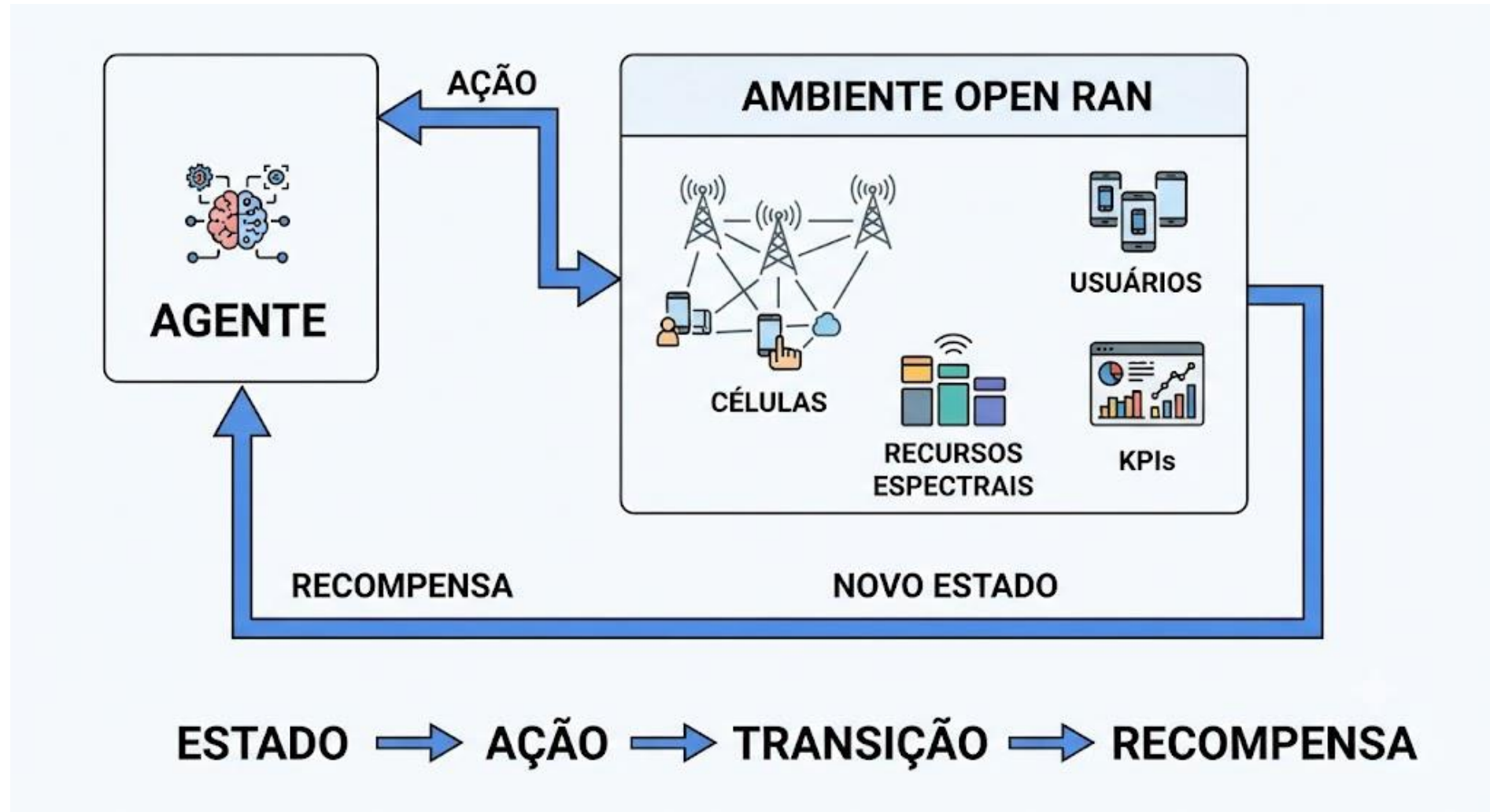
- ❑ MDP modela ambientes dinâmicos sequenciais.
- ❑ Estados representam situações possíveis.
- ❑ Ações representam decisões disponíveis.
- ❑ Recompensas orientam comportamento desejado.

MARKOV DECISION PROCESS - MDP

ELEMENTOS DO MDP

- ❑ Estado descreve condição atual ambiente.
- ❑ Ação representa escolha realizada agente.
- ❑ Transição modifica situação após ação.
- ❑ Recompensa avalia resultado obtido.

MARKOV DECISION PROCESS - MDP



MARKOV DECISION PROCESS - MDP

ESTADOS

- ❑ Estado representa situação operacional atual.
- ❑ Informações relevantes orientam decisões.
- ❑ Estados devem capturar contexto suficiente.
- ❑ Representação influencia qualidade aprendizado.

MARKOV DECISION PROCESS - MDP

AÇÕES

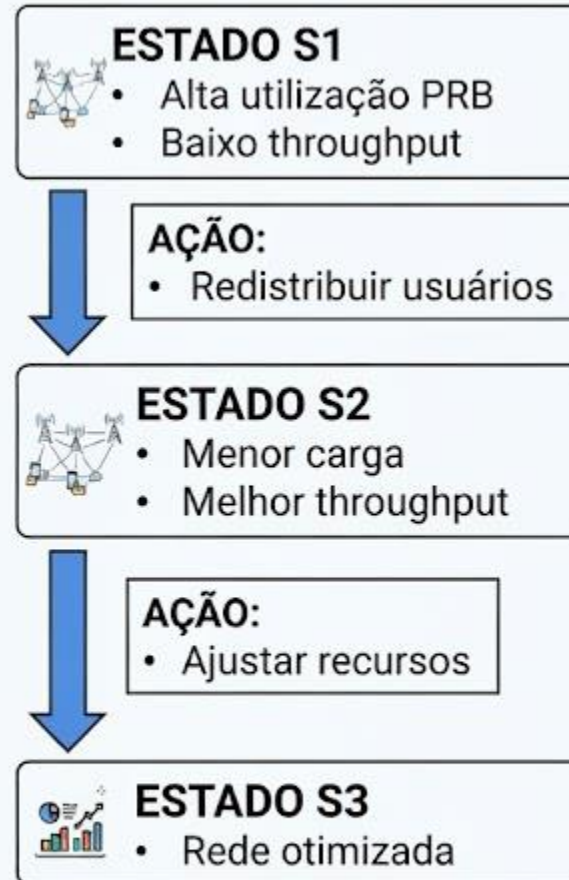
- ❑ Ações representam intervenções possíveis.
- ❑ Agente seleciona uma alternativa disponível.
- ❑ Espaço de ações define complexidade.
- ❑ Decisões alteram comportamento da rede.

MARKOV DECISION PROCESS - MDP

TRANSIÇÕES ENTRE ESTADOS

- ❑ Ações provocam mudanças ambientais.
- ❑ Novo estado depende decisão realizada.
- ❑ Ambientes possuem incertezas naturais.
- ❑ Agente aprende consequências das ações.

MARKOV DECISION PROCESS - MDP



AÇÕES transformam ESTADOS

MARKOV DECISION PROCESS - MDP

RECOMPENSA IMEDIATA OU FUTURA?

- ❑ Recompensa mede resultado da ação.
- ❑ Benefício imediato pode ser limitado.
- ❑ Decisões devem considerar futuro.
- ❑ Retorno acumulado orienta estratégia.

MARKOV DECISION PROCESS - MDP

RETORNO ACUMULADO

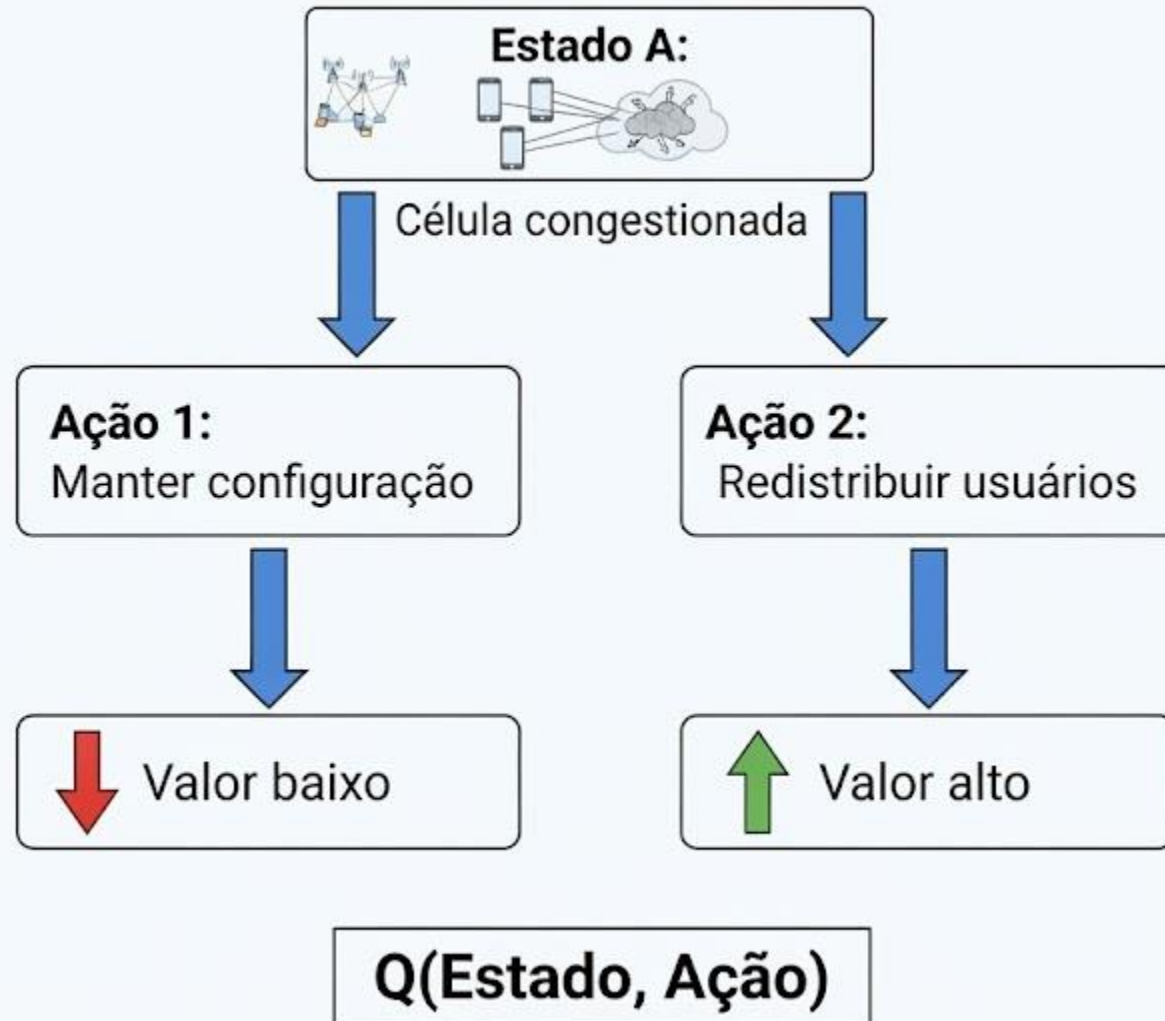
- ❑ Retorno soma recompensas futuras.
- ❑ Futuro recebe importância ajustável.
- ❑ Fator desconto controla horizonte temporal.
- ❑ Estratégias equilibram presente e futuro.

MARKOV DECISION PROCESS - MDP

FUNÇÃO DE VALOR

- ❑ Valor representa qualidade de estados.
- ❑ Estados melhores possuem maior retorno esperado.
- ❑ Agente estima benefícios futuros.
- ❑ Valores orientam decisões futuras.

MARKOV DECISION PROCESS - MDP



MARKOV DECISION PROCESS - MDP

CONCEITO DE Q-VALUE

- ❑ Q-value avalia ações específicas.
- ❑ Combina estado atual e ação.
- ❑ Valores maiores indicam melhores escolhas.
- ❑ Agente aprende tabela de decisões.

MARKOV DECISION PROCESS - MDP

ALGORITMO Q-LEARNING

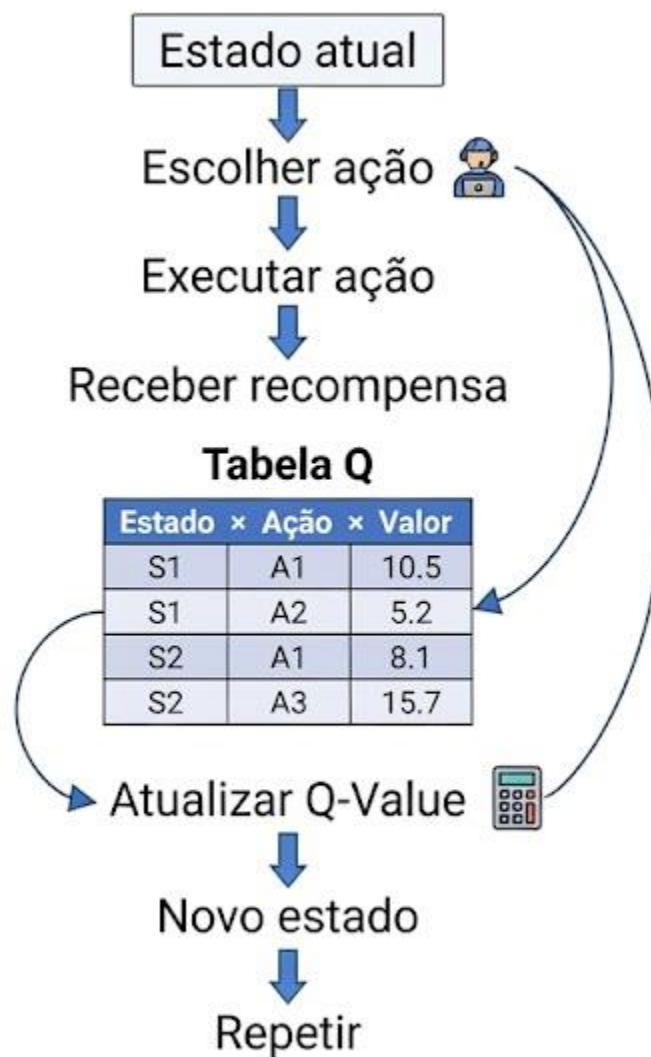
- ❑ Q-Learning aprende valores das ações.
- ❑ Experiência atualiza conhecimento acumulado.
- ❑ Melhor ação maximiza recompensa futura.
- ❑ Política emerge dos valores aprendidos.

MARKOV DECISION PROCESS - MDP

ATUALIZANDO O CONHECIMENTO

- ❑ Agente testa diferentes decisões.
- ❑ Resultados ajustam valores aprendidos.
- ❑ Erros corrigem estratégias anteriores.
- ❑ Experiência melhora desempenho gradualmente.

MARKOV DECISION PROCESS - MDP



MARKOV DECISION PROCESS - MDP

EXPLORAÇÃO VERSUS APROVEITAMENTO

- ❑ Agente precisa testar novas ações.
- ❑ Conhecimento existente também deve utilizado.
- ❑ Equilíbrio melhora aprendizado eficiente.
- ❑ Estratégia ϵ -greedy controla escolha.

MARKOV DECISION PROCESS - MDP

Q-LEARNING EM REDES MÓVEIS

- ❑ Estados representam condições da rede.
- ❑ Ações controlam recursos disponíveis.
- ❑ Recompensas refletem objetivos operacionais.
- ❑ Agente aprende políticas eficientes.

MARKOV DECISION PROCESS - MDP

LIMITAÇÃO DA TABELA Q

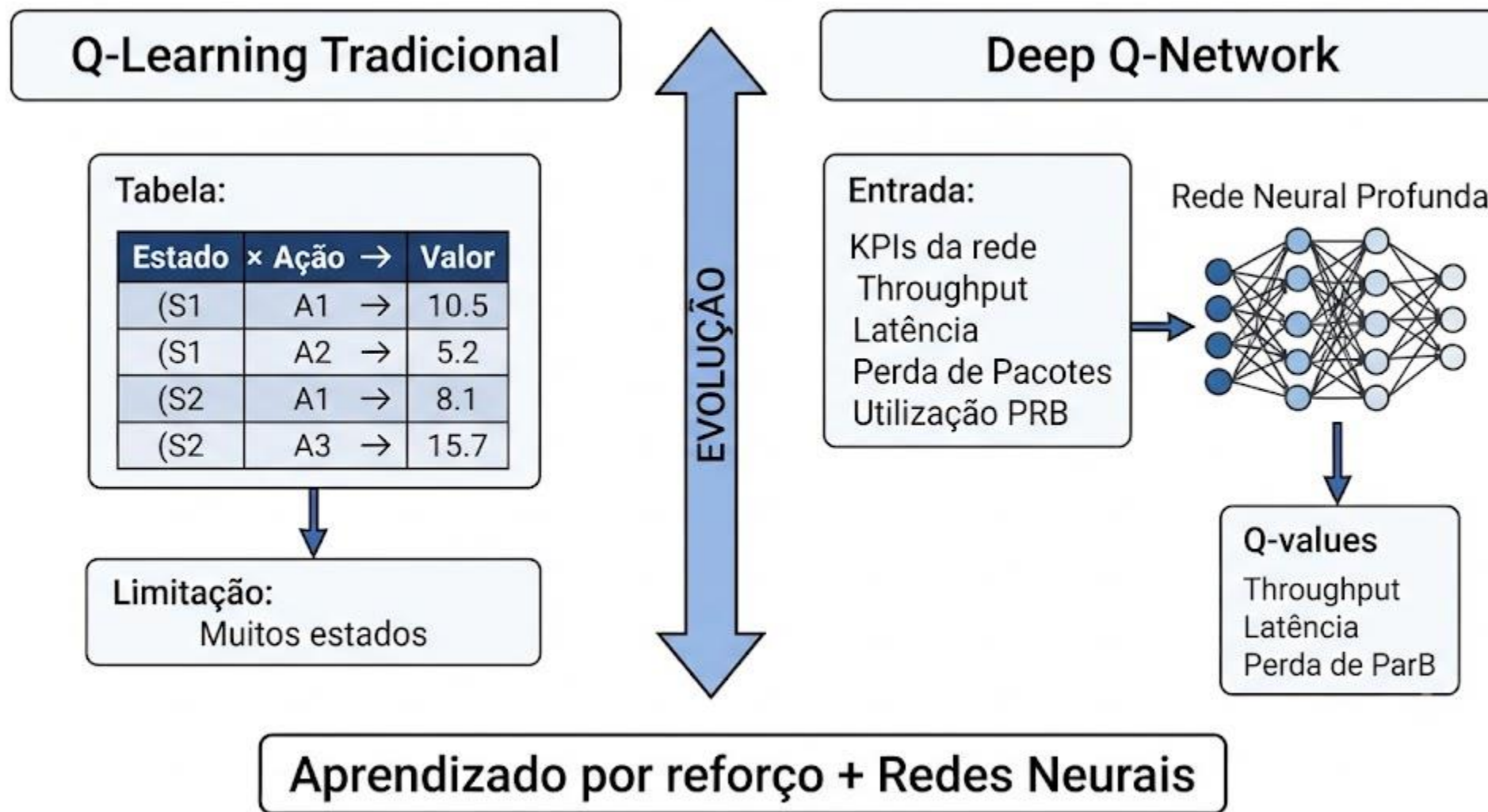
- ❑ Espaços grandes tornam tabela inviável.
- ❑ Redes reais possuem muitos estados.
- ❑ Variáveis contínuas aumentam complexidade.
- ❑ Redes neurais tornam-se necessárias.

MARKOV DECISION PROCESS - MDP

LIMITAÇÃO DA TABELA Q

- ❑ Espaços grandes tornam tabela inviável.
- ❑ Redes reais possuem muitos estados.
- ❑ Variáveis contínuas aumentam complexidade.
- ❑ Redes neurais tornam-se necessárias.

MARKOV DECISION PROCESS - MDP





C . E . S . A . R
school

especialização

Aplicações IA/ML RIC: Deep Q-Network – DQN

Julio Tesolin

Open RAN Redes Abertas e Tecnologias Emergentes

DEEP Q-NETWORK - DQN

O LIMITE DO Q-LEARNING

- ❑ Q-Learning utiliza tabelas de valores.
- ❑ Estados e ações crescem rapidamente.
- ❑ Redes reais possuem alta complexidade.
- ❑ Abordagem tradicional perde escalabilidade.

DEEP Q-NETWORK - DQN

EXPLOÇÃO DOS ESTADOS

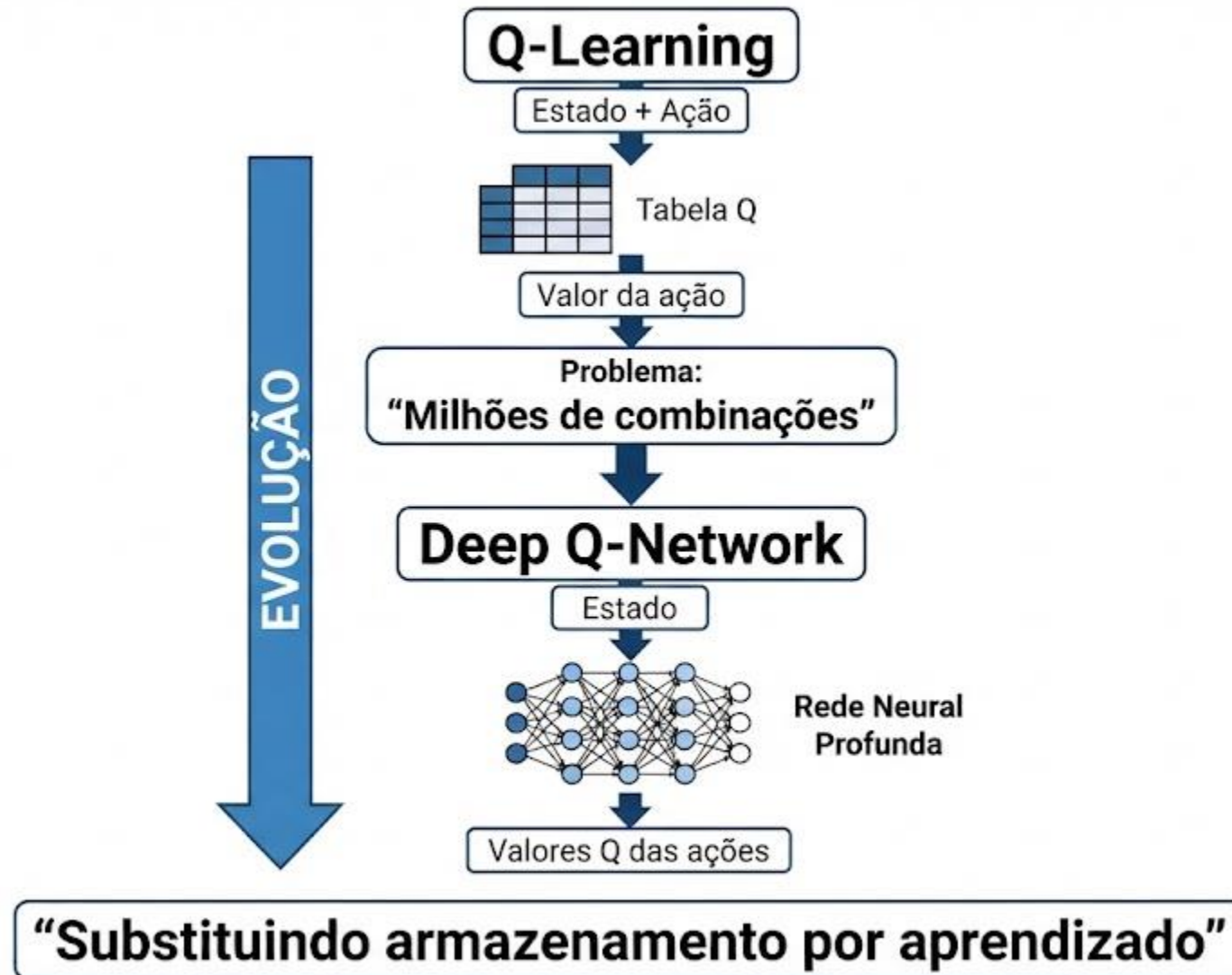
- ❑ Redes móveis possuem múltiplas variáveis.
- ❑ Estados combinam diferentes condições operacionais.
- ❑ Número de possibilidades cresce exponencialmente.
- ❑ Tabelas completas tornam-se impraticáveis.

DEEP Q-NETWORK - DQN

NECESSIDADE DAS REDES NEURAIAS

- ❑ Redes neurais identificam padrões complexos.
- ❑ Aproximações substituem tabelas gigantescas.
- ❑ Modelos generalizam conhecimento adquirido.
- ❑ DQN combina decisão e representação.

DEEP Q-NETWORK - DQN



DEEP Q-NETWORK - DQN

IDEIA CENTRAL DO DQN

- ❑ DQN aproxima função valor-ação.
- ❑ Estado alimenta rede neural profunda.
- ❑ Saída representa valores das ações.
- ❑ Maior valor indica melhor decisão.

DEEP Q-NETWORK - DQN

REDE NEURAL COMO FUNÇÃO Q

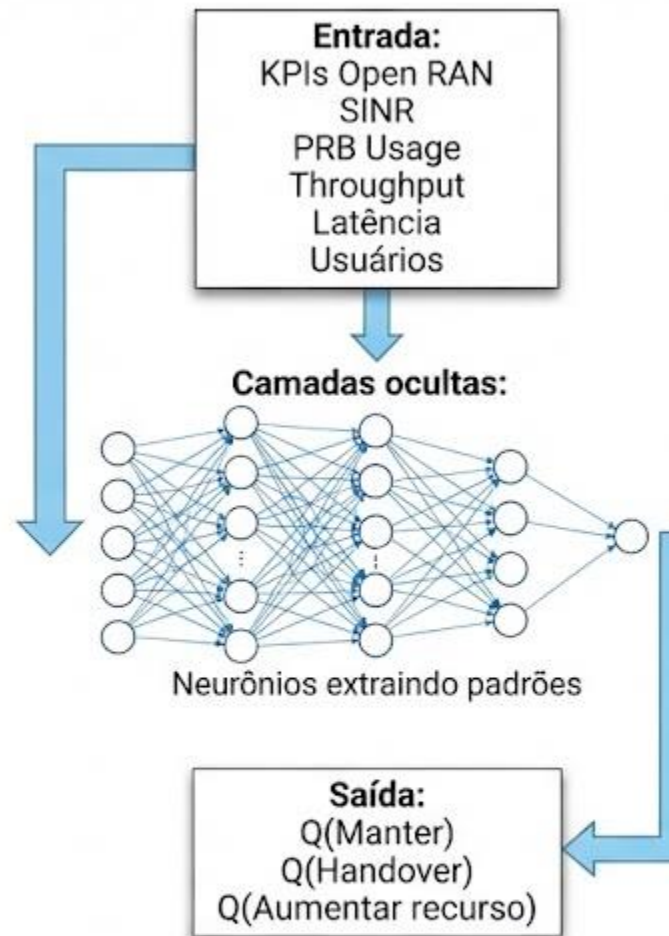
- ❑ Rede aprende relação estado-ação.
- ❑ Pesos armazenam conhecimento adquirido.
- ❑ Treinamento ajusta previsões de valor.
- ❑ Modelo melhora através da experiência.

DEEP Q-NETWORK - DQN

ARQUITETURA BÁSICA DA DQN

- ❑ Entrada representa estado observado.
- ❑ Camadas ocultas extraem padrões relevantes.
- ❑ Saída estima valores das ações.
- ❑ Decisão escolhe maior valor Q .

DEEP Q-NETWORK - DQN



Estado → Rede Neural → Valores Q

DEEP Q-NETWORK - DQN

ESTADOS COMO DADOS DE ENTRADA

- ❑ Estados representam contexto operacional.
- ❑ KPIs descrevem situação da rede.
- ❑ Dados precisam representar decisões relevantes.
- ❑ Qualidade influencia aprendizado final.

DEEP Q-NETWORK - DQN

SAÍDA REPRESENTA DECISÕES POSSÍVEIS

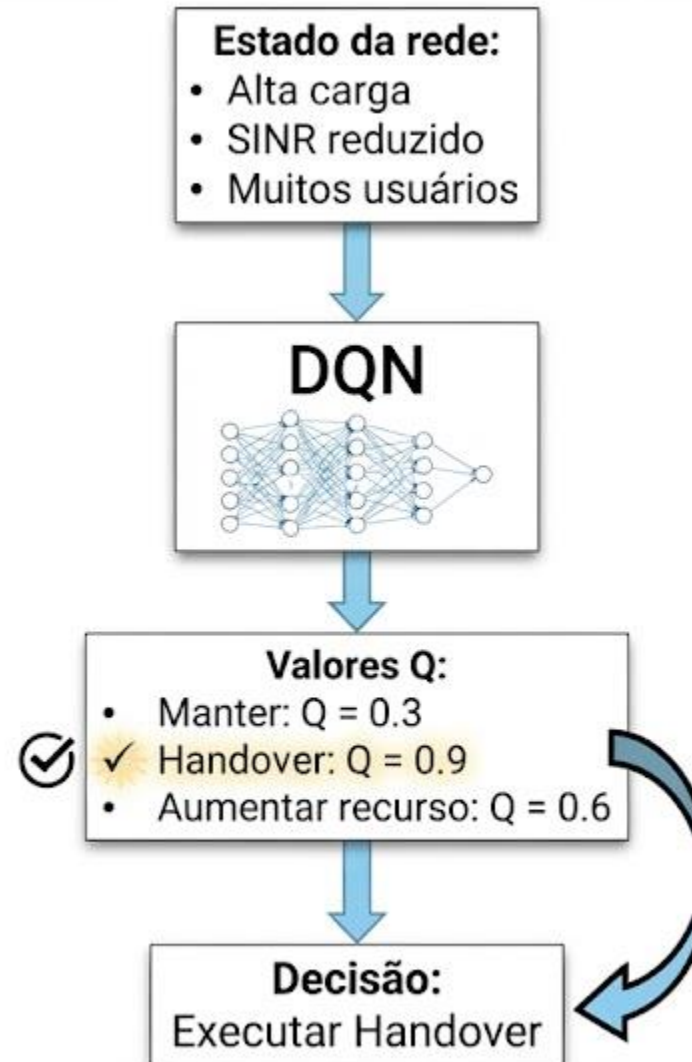
- ❑ Cada saída corresponde uma ação.
- ❑ Valores Q estimam benefícios futuros.
- ❑ Agente compara alternativas disponíveis.
- ❑ Maior Q orienta escolha final.

DEEP Q-NETWORK - DQN

ESCOLHENDO A MELHOR AÇÃO

- ❑ Agente avalia diferentes alternativas.
- ❑ Valores Q orientam comportamento.
- ❑ Melhor ação maximiza retorno esperado.
- ❑ Política emerge do treinamento.

DEEP Q-NETWORK - DQN



DEEP Q-NETWORK - DQN

DESAFIO DA INSTABILIDADE

- ❑ Treinamento possui dados correlacionados.
- ❑ Ambiente muda continuamente.
- ❑ Atualizações podem oscilar.
- ❑ Técnicas estabilizam aprendizado.

DEEP Q-NETWORK - DQN

EXPERIENCE REPLAY

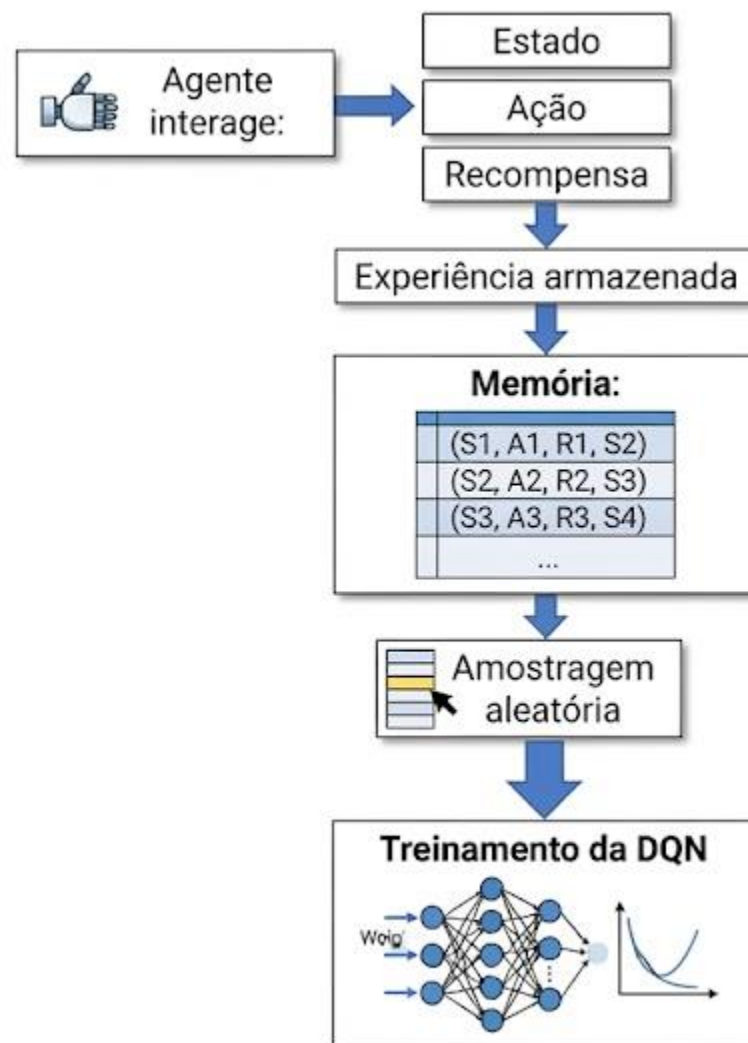
- ❑ Experiências são armazenadas temporariamente.
- ❑ Memória reúne situações anteriores.
- ❑ Amostras aleatórias treinam rede.
- ❑ Aprendizado torna-se mais estável.

DEEP Q-NETWORK - DQN

BENEFÍCIOS DA MEMÓRIA DE EXPERIÊNCIAS

- ❑ Reduz dependência temporal dos dados.
- ❑ Melhora eficiência do treinamento.
- ❑ Reutiliza experiências importantes.
- ❑ Aumenta capacidade de generalização.

DEEP Q-NETWORK - DQN



DEEP Q-NETWORK - DQN

TARGET NETWORK

- ❑ Rede auxiliar estabiliza treinamento.
- ❑ Valores objetivos ficam mais consistentes.
- ❑ Atualizações ocorrem periodicamente.
- ❑ Convergência torna-se mais confiável.

DEEP Q-NETWORK - DQN

TREINAMENTO COMPLETO DA DQN

- ❑ Estado alimenta rede neural.
- ❑ Ação selecionada pelo agente.
- ❑ Experiência atualiza conhecimento.
- ❑ Rede melhora progressivamente.

DEEP Q-NETWORK - DQN

DQN NO TRAFFIC STEERING

- ❑ Estado inclui condições celulares.
- ❑ Ações controlam mobilidade usuários.
- ❑ Recompensa mede qualidade serviço.
- ❑ Agente aprende distribuição eficiente.

DEEP Q-NETWORK - DQN

LIMITAÇÕES DO DQN

- ❑ Atua principalmente ações discretas.
- ❑ Treinamento exige muitos dados.
- ❑ Exploração pode ser ineficiente.
- ❑ Ambientes complexos exigem novas abordagens.

DEEP Q-NETWORK - DQN

Q-Learning 

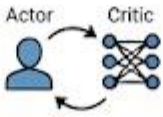


Deep Q-Network 



Policy Gradient 



Actor-Critic 

DQN aprende:
"Qual ação possui maior valor?"

Actor-Critic aprende:
"Qual política deve ser seguida?"

PRATICA KNIME

TIC TAC TOE – DQN LEARNING

<https://hub.knime.com/corey/spaces/Public/Tic-Tac-Toe/Tic-Tac-Toe%20Learn~pjN-0Sm6RtZ3b3Hl/current-state>



especialização

Aplicações IA/ML RIC: Policy Gradient & Actor-Critic

Julio Tesolin

Open RAN Redes Abertas e Tecnologias Emergentes

POLICY GRADIENT & ACTOR-CRITIC

LIMITE DO DQN

- ❑ DQN aprende valores das ações.
- ❑ Decisões dependem dos valores estimados.
- ❑ Ações contínuas apresentam dificuldades práticas.
- ❑ Novas abordagens aprendem políticas diretamente.

POLICY GRADIENT & ACTOR-CRITIC

DUAS FILOSOFIAS DO RL

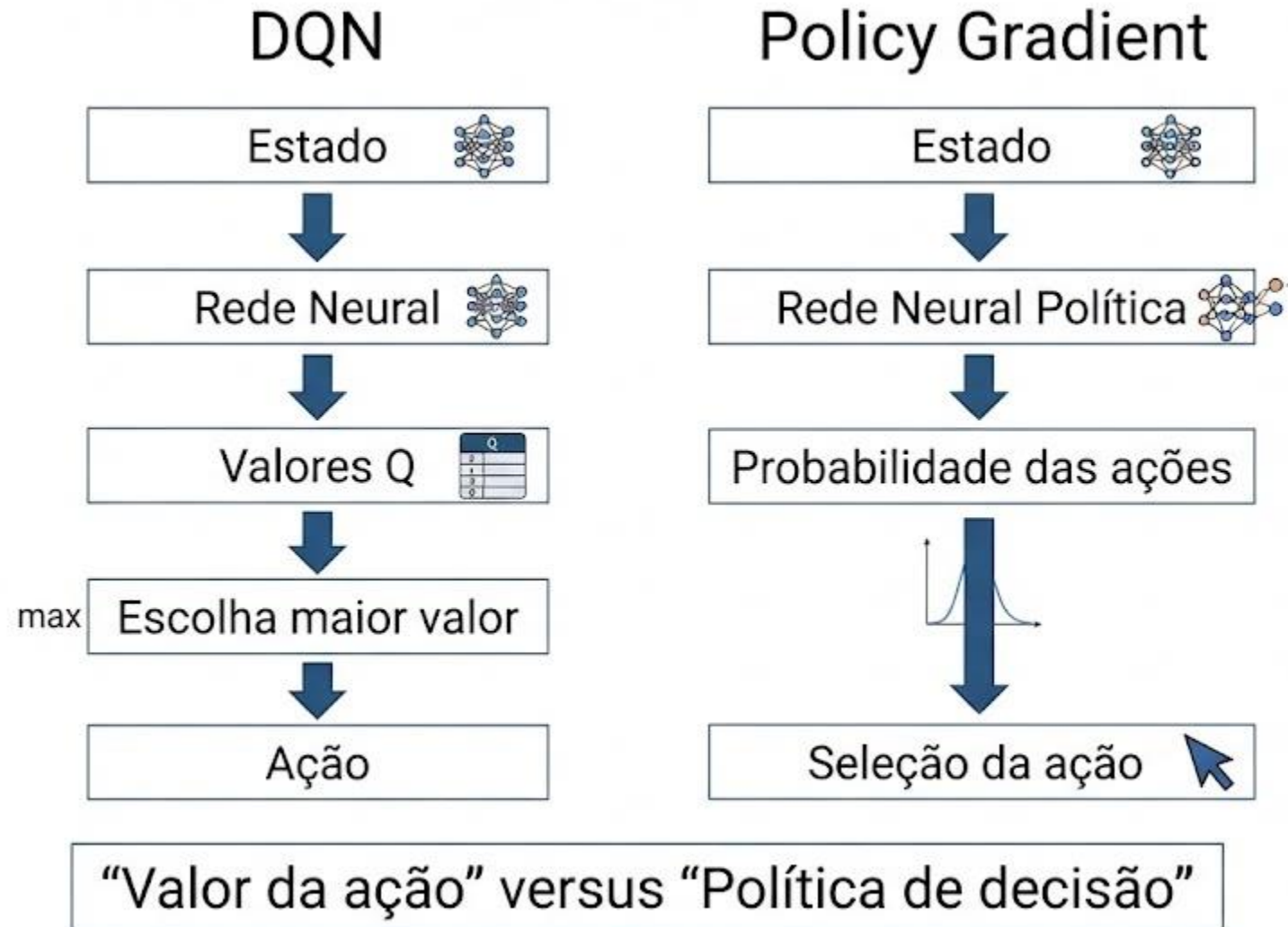
- ❑ Valor avalia qualidade das ações.
- ❑ Política define comportamento do agente.
- ❑ Métodos possuem objetivos diferentes.
- ❑ Ambos buscam maximizar recompensas.

POLICY GRADIENT & ACTOR-CRITIC

VALUE-BASED x POLICY-BASED

- ❑ Value-Based estima valores futuros.
- ❑ Policy-Based aprende decisões diretamente.
- ❑ DQN pertence aos métodos Value-Based.
- ❑ Actor-Critic combina ambas abordagens.

POLICY GRADIENT & ACTOR-CRITIC



POLICY GRADIENT & ACTOR-CRITIC

CONCEITO DE POLÍTICA

- ❑ Política define comportamento do agente.
- ❑ Relaciona estados com ações possíveis.
- ❑ Pode representar escolhas probabilísticas.
- ❑ Evolui durante o aprendizado.

POLICY GRADIENT & ACTOR-CRITIC

POLÍTICA DETERMINÍSTICA E PROBABILÍSTICA

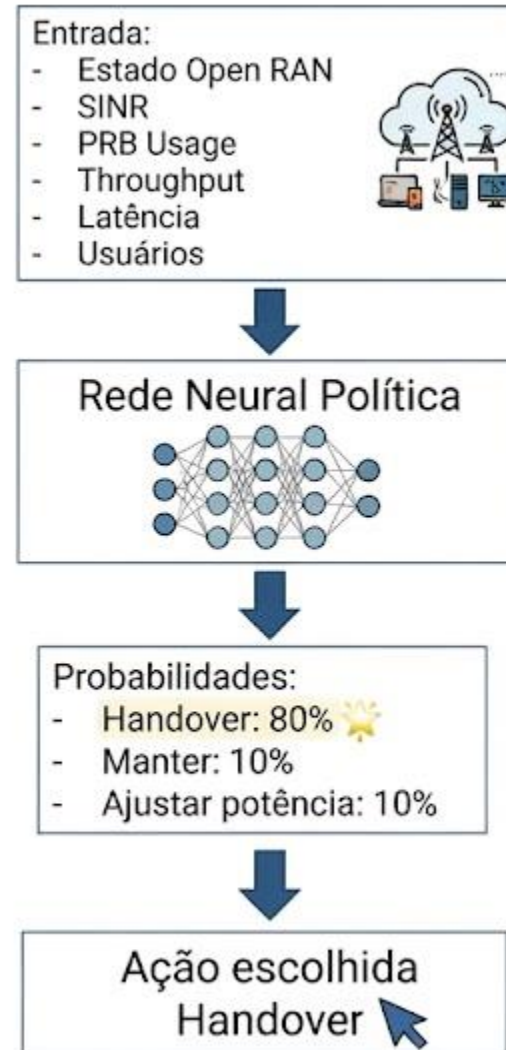
- ❑ Política determinística escolhe uma ação.
- ❑ Política probabilística distribui possibilidades.
- ❑ Exploração depende da política adotada.
- ❑ Flexibilidade aumenta capacidade adaptativa.

POLICY GRADIENT & ACTOR-CRITIC

REDE NEURAL COMO POLÍTICA

- ❑ Rede representa estratégia aprendida.
- ❑ Entrada recebe estado observado.
- ❑ Saída gera ações possíveis.
- ❑ Pesos armazenam comportamento adquirido.

POLICY GRADIENT & ACTOR-CRITIC



POLICY GRADIENT & ACTOR-CRITIC

PRINCÍPIO DO POLICY GRADIENT

- ❑ Política recebe ajustes pela recompensa.
- ❑ Boas ações aumentam probabilidade.
- ❑ Más ações reduzem probabilidade.
- ❑ Aprendizado ocorre pela experiência.

POLICY GRADIENT & ACTOR-CRITIC

APRENDENDO PELAS RECOMPENSAS

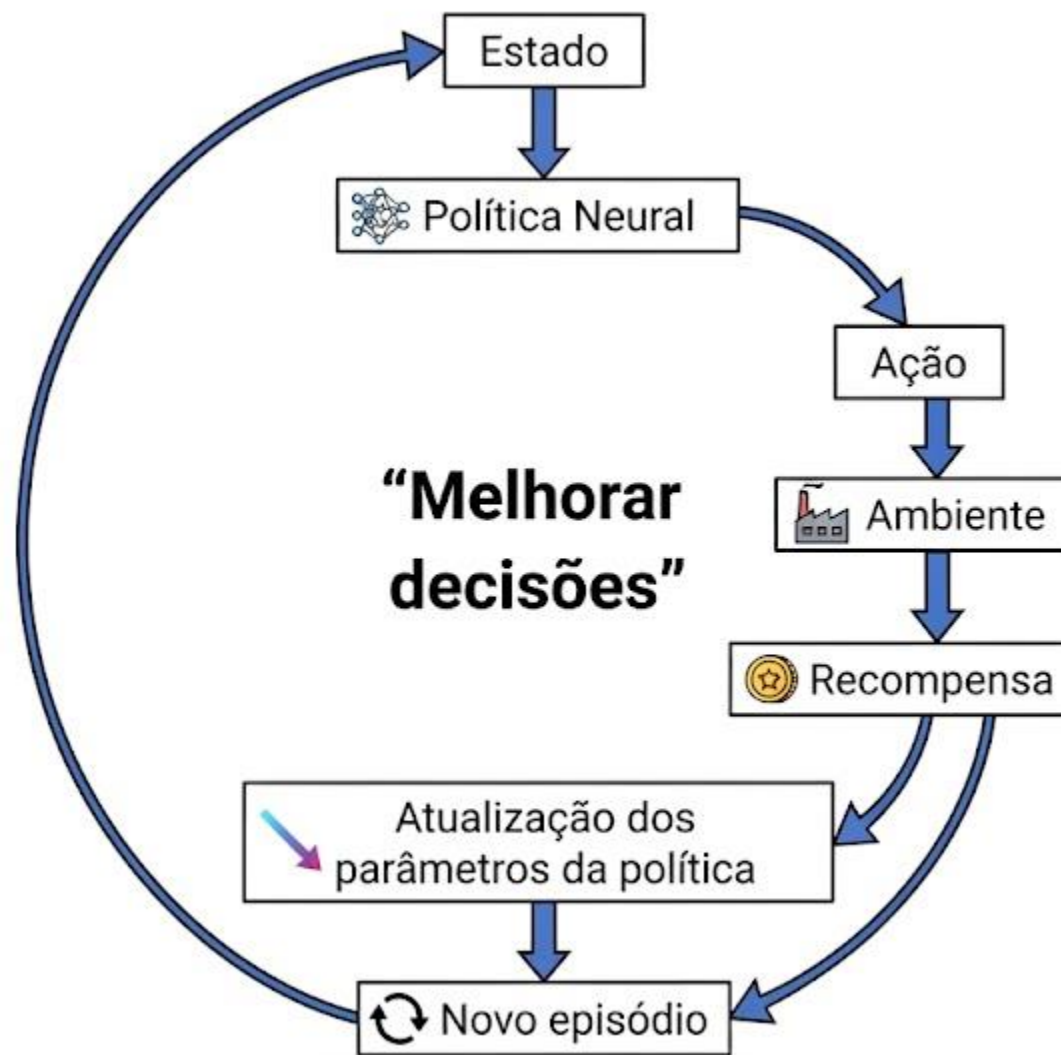
- ❑ Agente executa trajetórias completas.
- ❑ Recompensas avaliam decisões realizadas.
- ❑ Política é ajustada continuamente.
- ❑ Melhor comportamento emerge gradualmente.

POLICY GRADIENT & ACTOR-CRITIC

ALGORITMO REINFORCE

- ❑ Primeiro método Policy Gradient clássico.
- ❑ Aprende diretamente uma política.
- ❑ Usa retorno acumulado das ações.
- ❑ Atualiza parâmetros da rede neural.

POLICY GRADIENT & ACTOR-CRITIC



POLICY GRADIENT & ACTOR-CRITIC

LIMITAÇÕES DO POLICY GRADIENT

- ❑ Aprendizado possui alta variabilidade.
- ❑ Muitas experiências são necessárias.
- ❑ Atualizações podem ser instáveis.
- ❑ Avaliação intermediária torna-se necessária.

POLICY GRADIENT & ACTOR-CRITIC

MOTIVAÇÃO DO ACTOR-CRITIC

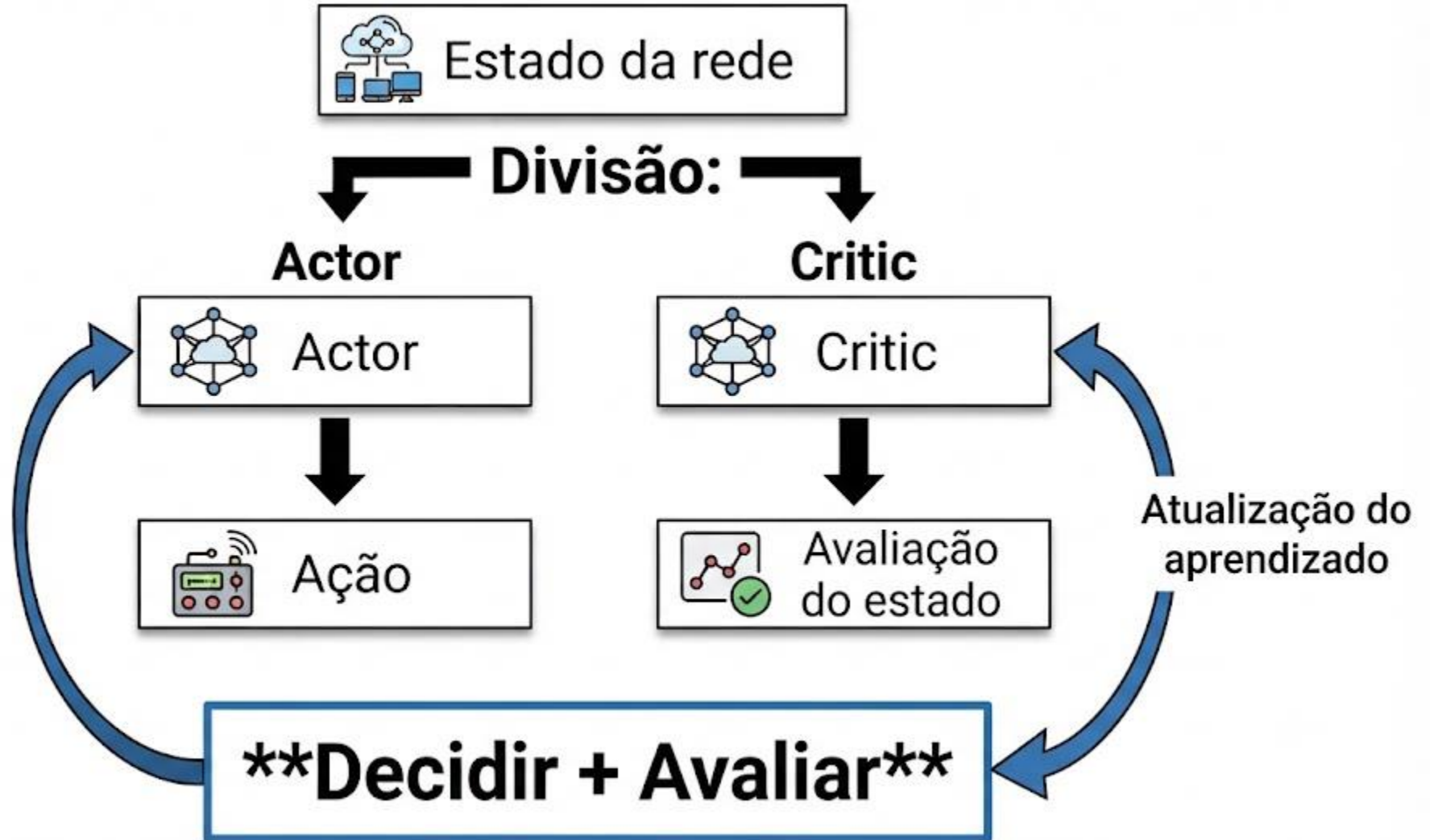
- ❑ Combina política e avaliação.
- ❑ Actor escolhe ações.
- ❑ Critic avalia decisões.
- ❑ Aprendizado torna-se mais estável.

POLICY GRADIENT & ACTOR-CRITIC

ARQUITETURA ACTOR-CRITIC

- ❑ Actor representa política neural.
- ❑ Critic estima valor esperado.
- ❑ Ambos aprendem simultaneamente.
- ❑ Estrutura melhora estabilidade.

POLICY GRADIENT & ACTOR-CRITIC



POLICY GRADIENT & ACTOR-CRITIC

PAPEL DO ACTOR

- ❑ Actor gera ações possíveis.
- ❑ Política controla comportamento aprendido.
- ❑ Decisões dependem estado atual.
- ❑ Rede melhora com experiência.

POLICY GRADIENT & ACTOR-CRITIC

PAPEL DO CRITIC

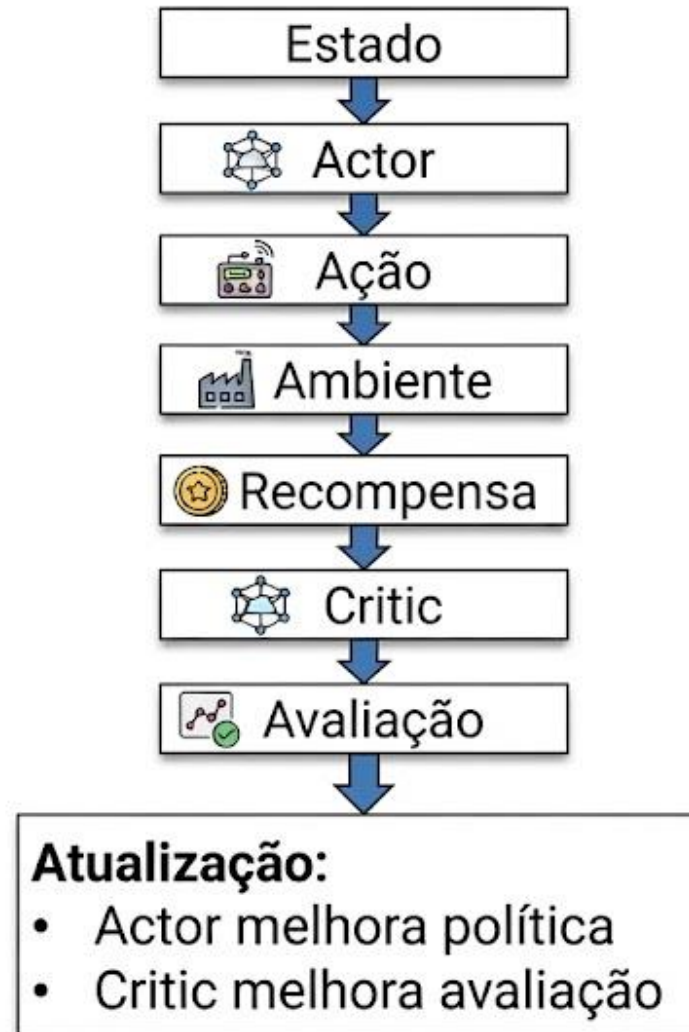
- ❑ Critic avalia qualidade das ações.
- ❑ Estima valor esperado futuro.
- ❑ Orienta atualização do Actor.
- ❑ Reduz incerteza no aprendizado.

POLICY GRADIENT & ACTOR-CRITIC

CONCEITO DE ADVANTAGE

- ❑ Mede benefício adicional da ação.
- ❑ Compara ação com expectativa.
- ❑ Orienta melhorias da política.
- ❑ Acelera aprendizado do agente.

POLICY GRADIENT & ACTOR-CRITIC



POLICY GRADIENT & ACTOR-CRITIC

PRINCIPAIS ALGORITMOS ACTOR-CRITIC

- ❑ A2C utiliza aprendizado sincronizado.
- ❑ A3C explora múltiplos agentes.
- ❑ DDPG trabalha ações contínuas.
- ❑ PPO melhora estabilidade geral.

POLICY GRADIENT & ACTOR-CRITIC

PROXIMAL POLICY OPTIMIZATION (PPO)

- ❑ PPO controla mudanças da política.
- ❑ Evita atualizações excessivas.
- ❑ Melhora estabilidade treinamento.
- ❑ Aplicação ampla em sistemas complexos.



C . E . S . A . R
school

especialização

Aplicações IA/ML RIC: Algoritmos Bio-Inspirados

Julio Tesolin

Open RAN Redes Abertas e Tecnologias Emergentes

ALGORITMOS BIO-INSPIRADOS

O DESAFIO DA OTIMIZAÇÃO

- ❑ Redes modernas exigem decisões extremamente complexas
- ❑ Problemas possuem inúmeras soluções possíveis simultaneamente
- ❑ Métodos tradicionais nem sempre encontram boas respostas
- ❑ Otimização torna-se elemento essencial da inteligência

ALGORITMOS BIO-INSPIRADOS

PROBLEMAS COMPLEXOS DA ENGENHARIA

- ❑ Muitas variáveis interagem simultaneamente entre si
- ❑ Objetivos frequentemente apresentam conflitos operacionais importantes
- ❑ Restrições limitam soluções tecnicamente viáveis disponíveis
- ❑ Complexidade cresce rapidamente com tamanho problema

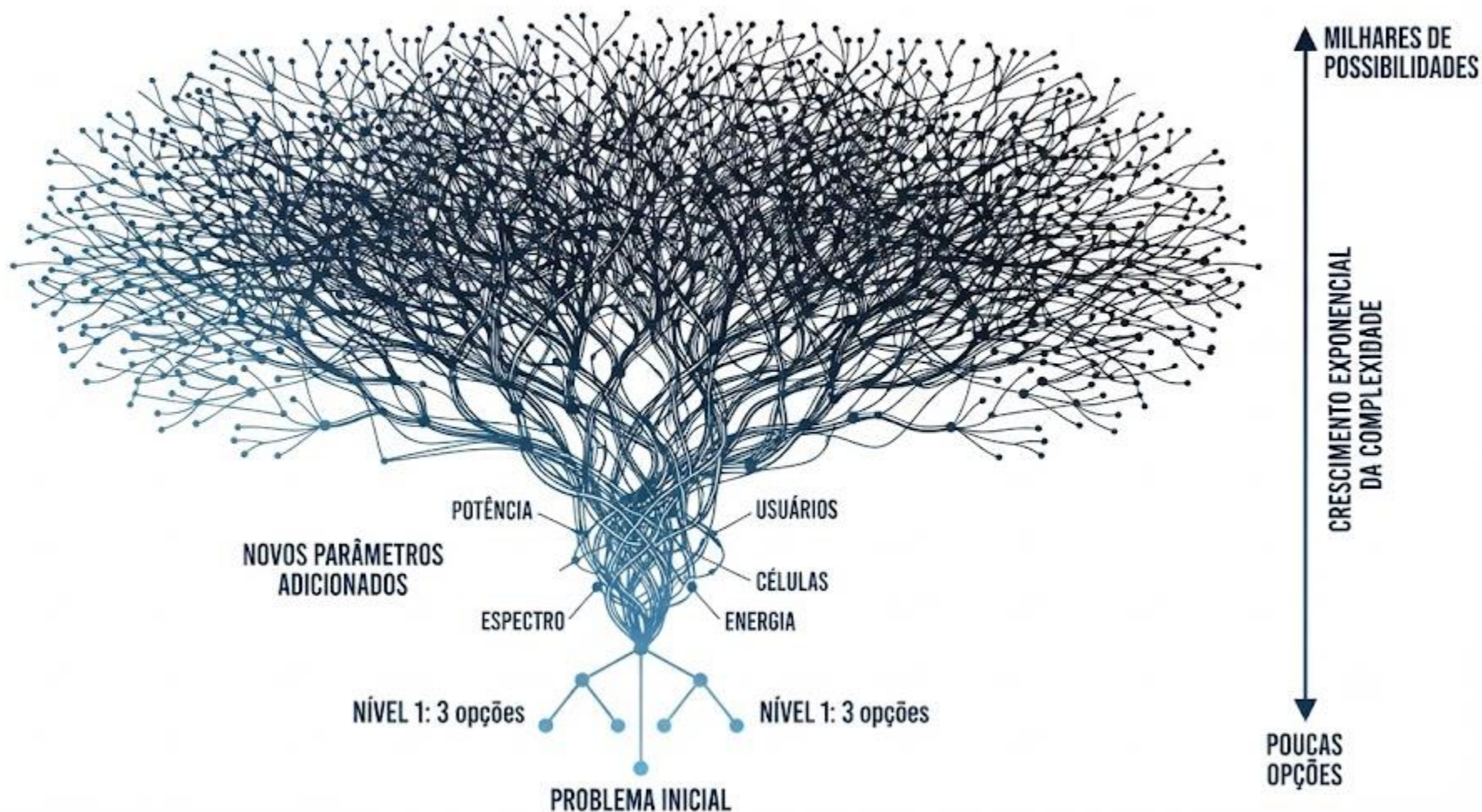
ALGORITMOS BIO-INSPIRADOS

LIMITAÇÕES DOS MÉTODOS TRADICIONAIS

- ❑ Métodos exatos exigem hipóteses matemáticas restritivas
- ❑ Problemas reais raramente seguem modelos simplificados
- ❑ Ótimos globais tornam-se difíceis encontrar computacionalmente
- ❑ Alternativas inteligentes passam ser necessárias

ALGORITMOS BIO-INSPIRADOS

EXPLORAR TODAS AS SOLUÇÕES TORNA-SE INVIÁVEL.



ALGORITMOS BIO-INSPIRADOS

NATUREZA COMO INSPIRAÇÃO

- ❑ Natureza resolve problemas extremamente complexos continuamente
- ❑ Evolução produz soluções altamente eficientes naturalmente
- ❑ Organismos adaptam comportamentos conforme ambiente muda
- ❑ Computação aproveita princípios observados biologicamente

ALGORITMOS BIO-INSPIRADOS

O QUE SÃO ALGORITMOS BIO-INSPIRADOS

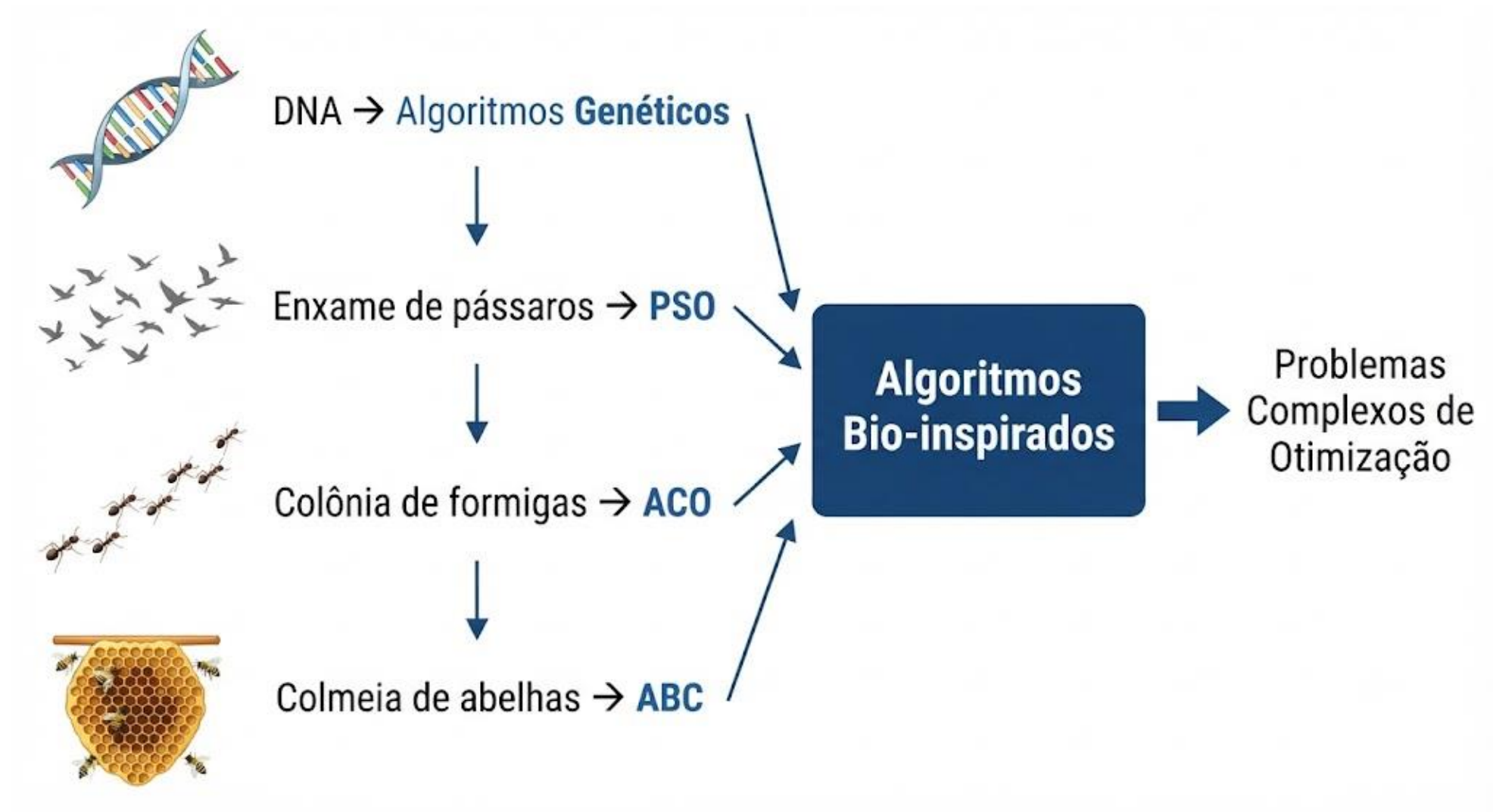
- ❑ Inspiram-se em fenômenos observados na natureza
- ❑ Buscam soluções aproximadas para otimização complexa
- ❑ Utilizam estratégias adaptativas durante execução
- ❑ Não reproduzem exatamente sistemas biológicos reais

ALGORITMOS BIO-INSPIRADOS

CARACTERÍSTICAS COMPARTILHADAS

- ❑ Busca ocorre através sucessivas iterações adaptativas
- ❑ Soluções evoluem progressivamente durante processamento
- ❑ Exploração equilibra intensificação das melhores soluções
- ❑ Robustez permite resolver problemas difíceis variados

ALGORITMOS BIO-INSPIRADOS



ALGORITMOS BIO-INSPIRADOS

EXPLORAÇÃO E INTENSIFICAÇÃO

- ❑ Exploração procura novas regiões promissoras continuamente
- ❑ Intensificação melhora soluções já descobertas anteriormente
- ❑ Equilíbrio evita convergência prematura do algoritmo
- ❑ Busca eficiente combina ambas estratégias simultaneamente

ALGORITMOS BIO-INSPIRADOS

HEURÍSTICAS E META-HEURÍSTICAS

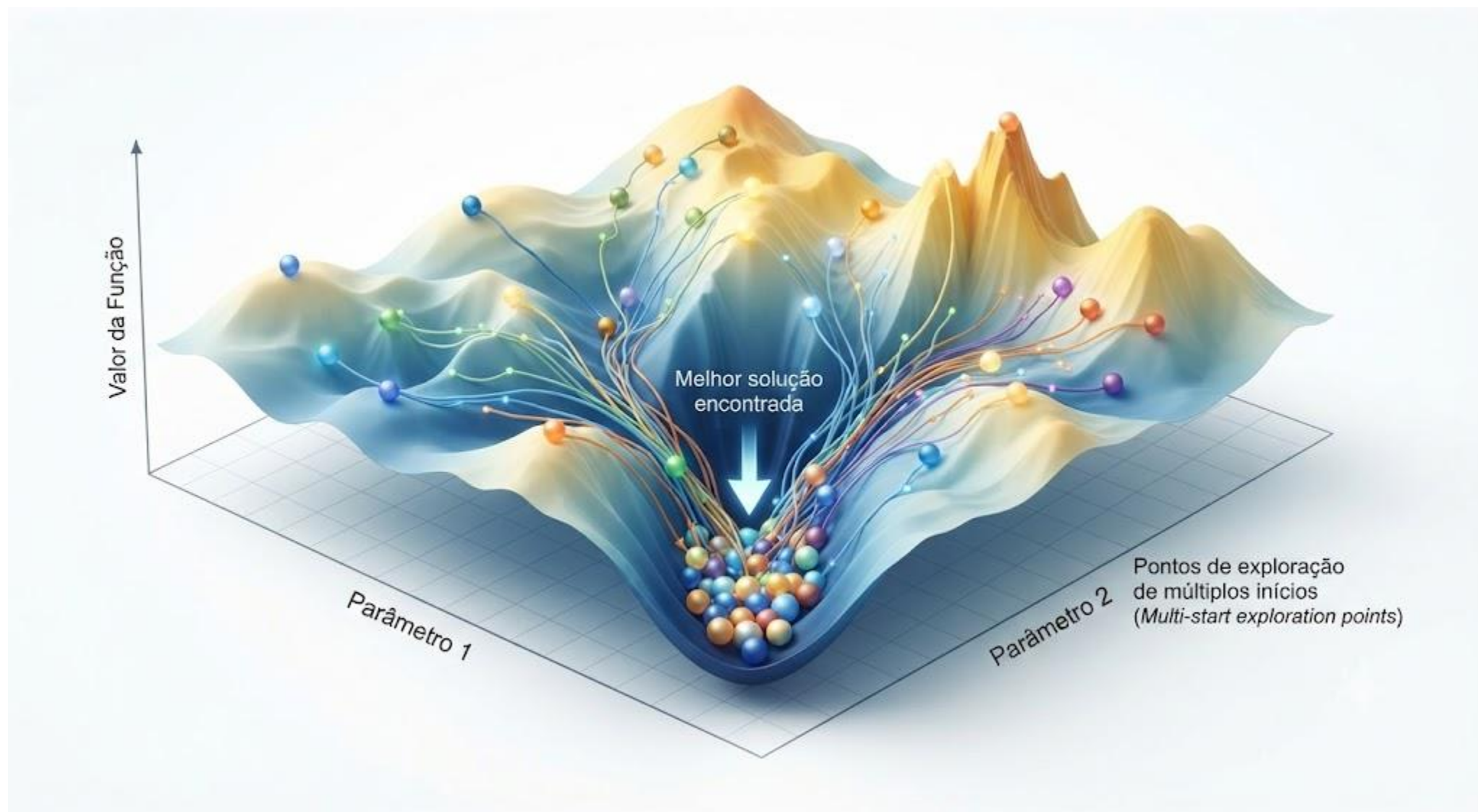
- ❑ Heurísticas resolvem problemas específicos rapidamente
- ❑ Meta-heurísticas atendem diferentes problemas complexos genericamente
- ❑ **Bio-inspirados** pertencem categoria das **meta-heurísticas**
- ❑ Flexibilidade amplia aplicações práticas importantes

ALGORITMOS BIO-INSPIRADOS

SOLUÇÕES APROXIMADAS

- ❑ Melhor solução nem sempre é encontrada
- ❑ Objetivo consiste obter solução suficientemente boa
- ❑ Tempo computacional reduz significativamente processamento
- ❑ Engenharia frequentemente privilegia eficiência prática

ALGORITMOS BIO-INSPIRADOS



ALGORITMOS BIO-INSPIRADOS

PRINCIPAIS FAMÍLIAS

- ❑ Algoritmos Genéticos simulam evolução biológica natural
- ❑ PSO inspira-se comportamento coletivo dos enxames
- ❑ ACO utiliza cooperação observada entre formigas
- ❑ Outros algoritmos exploram diferentes fenômenos naturais

ALGORITMOS BIO-INSPIRADOS

PROBLEMAS ADEQUADOS

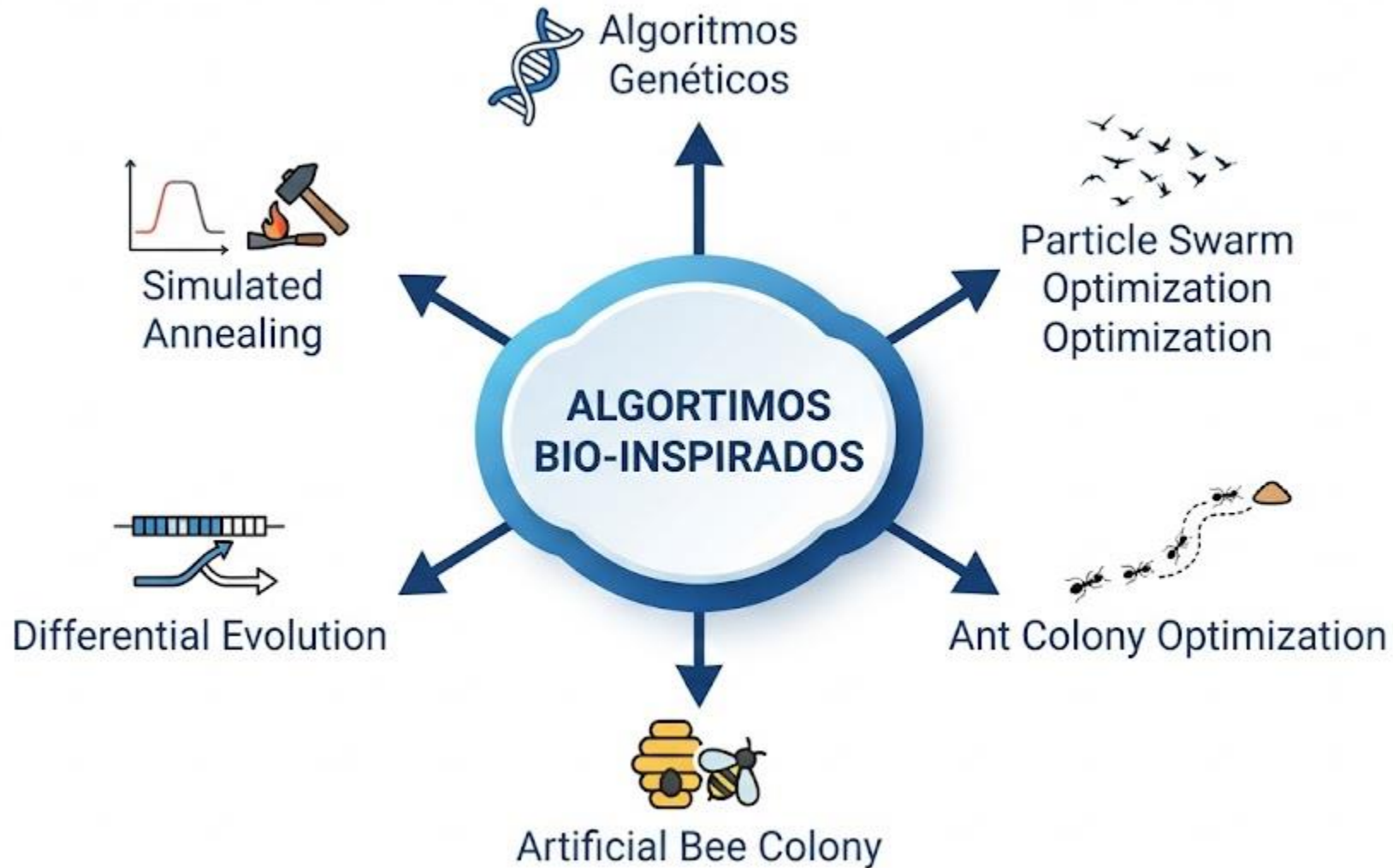
- ❑ Grandes espaços de busca combinatória
- ❑ Objetivos múltiplos frequentemente conflitantes simultaneamente
- ❑ Restrições tornam otimização extremamente difícil
- ❑ Soluções aproximadas apresentam excelente desempenho

ALGORITMOS BIO-INSPIRADOS

VANTAGENS E LIMITAÇÕES

- ❑ Adaptam-se facilmente diferentes tipos problemas
- ❑ Poucas hipóteses matemáticas são necessárias
- ❑ Resultados dependem configuração adequada parâmetros
- ❑ Garantias ótimas normalmente não existem

ALGORITMOS BIO-INSPIRADOS



ALGORITMOS BIO-INSPIRADOS

BIO-INSPIRADOS E INTELIGÊNCIA ARTIFICIAL

- ❑ Otimização complementa técnicas tradicionais inteligência artificial
- ❑ Redes neurais podem ser otimizadas automaticamente
- ❑ Aprendizado reforço beneficia-se otimização eficiente
- ❑ Integração amplia desempenho sistemas inteligentes

ALGORITMOS BIO-INSPIRADOS

QUANDO UTILIZAR

- ❑ Problemas apresentam elevada complexidade combinatória
- ❑ Métodos tradicionais tornam-se computacionalmente inviáveis
- ❑ Soluções aproximadas atendem requisitos operacionais plenamente
- ❑ Flexibilidade supera restrições modelos clássicos



C . E . S . A . R
school

especialização

Aplicações IA/ML RIC: Algoritmos Genéticos

Julio Tesolin

Open RAN Redes Abertas e Tecnologias Emergentes